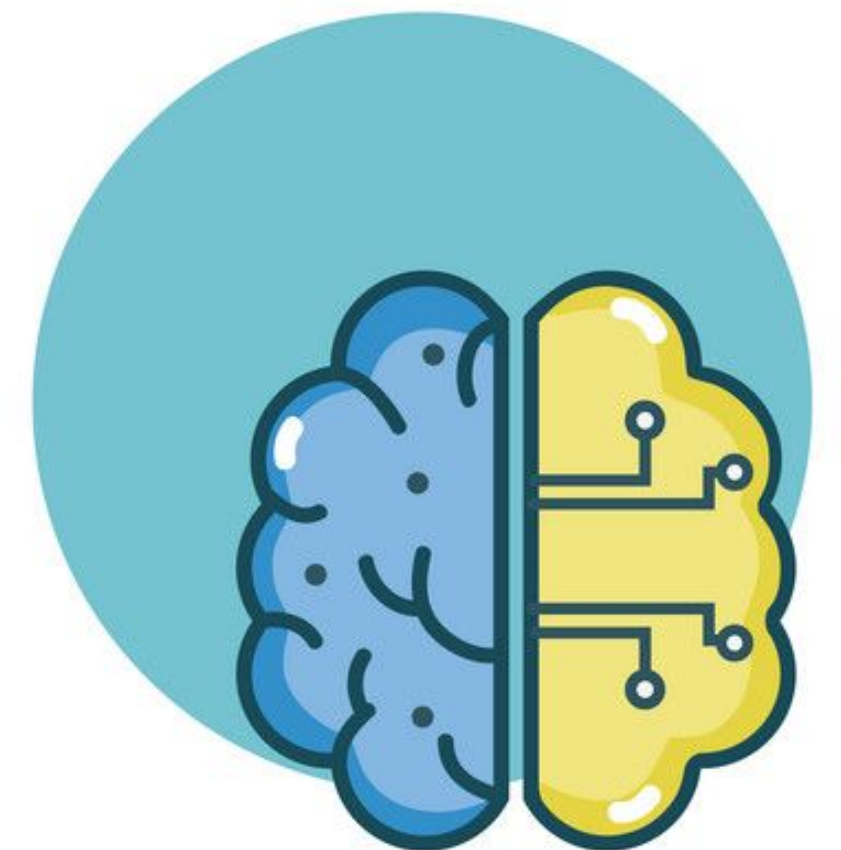


INTRODUCTION TO MACHINE LEARNING

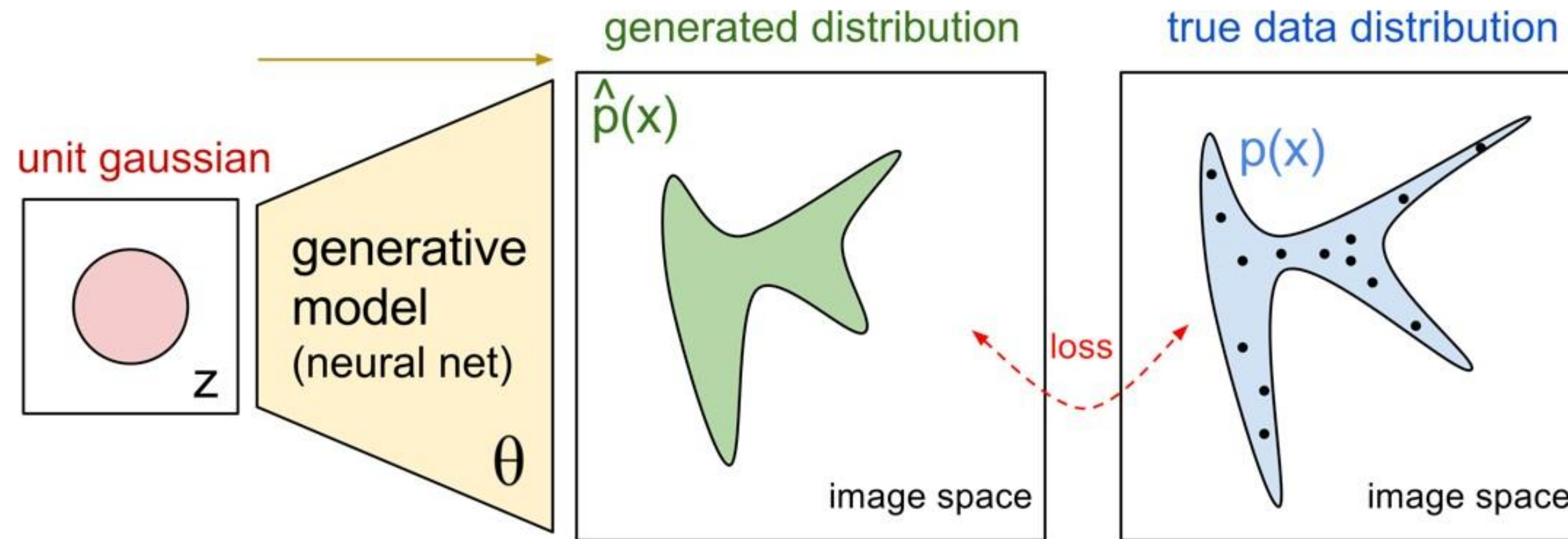
DIFFUSION MODELS



Elisa Ricci



GENERATIVE MODELS

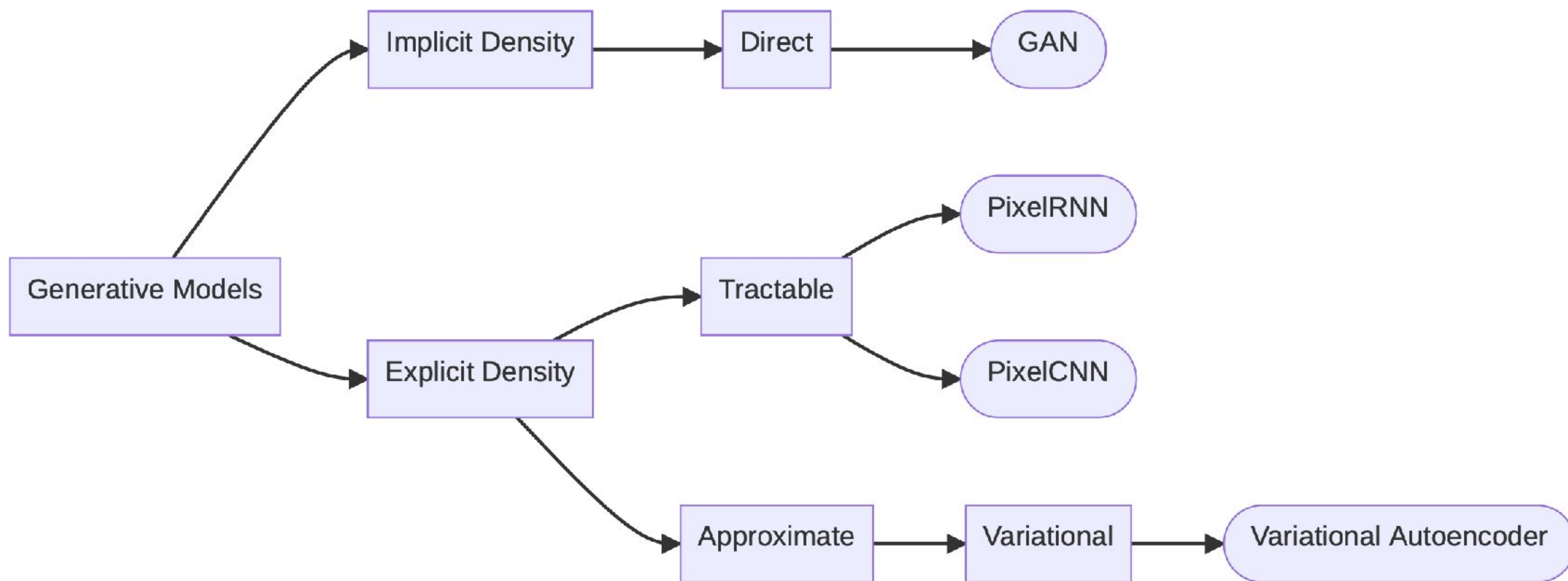


Train a generator G from latent space to data space and approximate the real data distribution

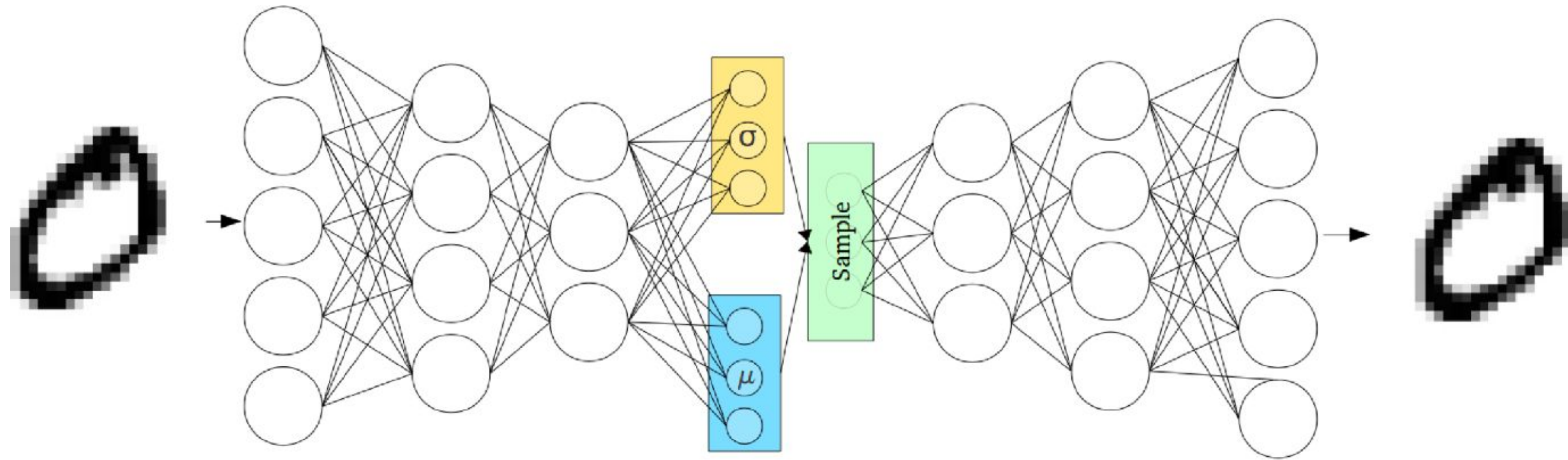
Training is difficult:

- Hyperparameters choice
- Quantify similarity between sets
- Choice of latent space

GENERATIVE MODELS

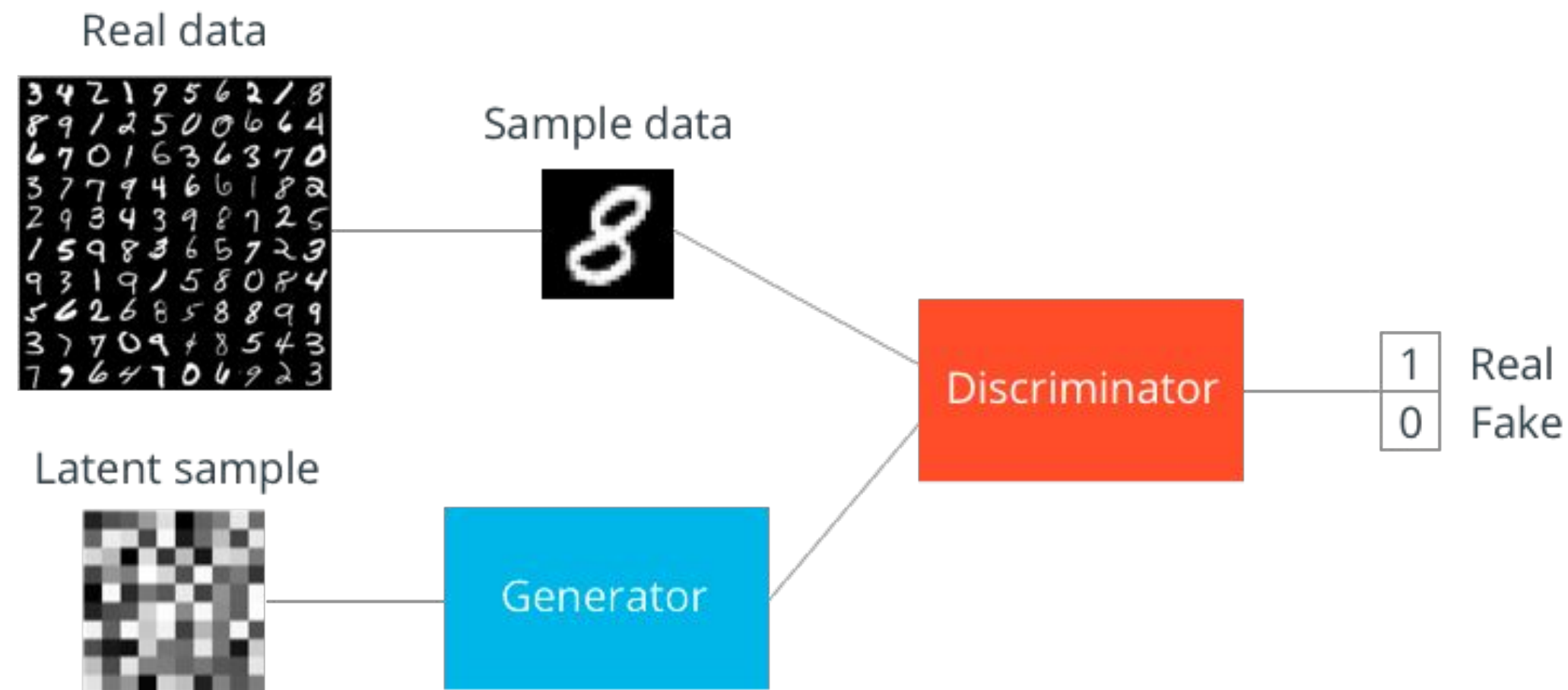


VARIATIONAL AUTOENCODERS



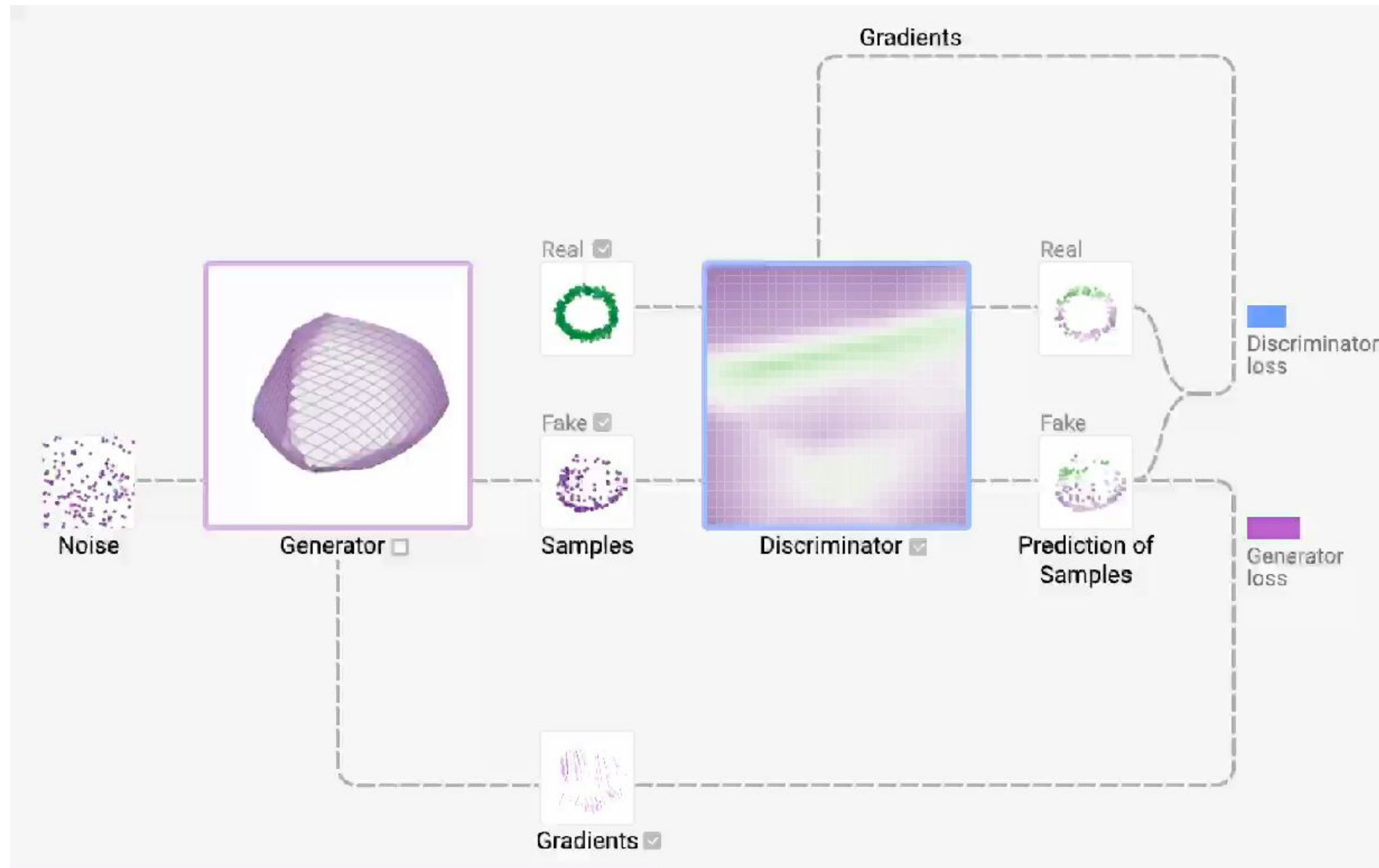
- Add to autoencoders the ability to generate new samples
 - Probabilistic latent space (assumed to be multivariate Gaussian)
- Minimize at the same time reconstruction error and KL divergence between latent and $N(0, \mathbf{I})$
- Sampling from the latent distribution allows generation and interpolation

GENERATIVE ADVERSARIAL NETWORKS

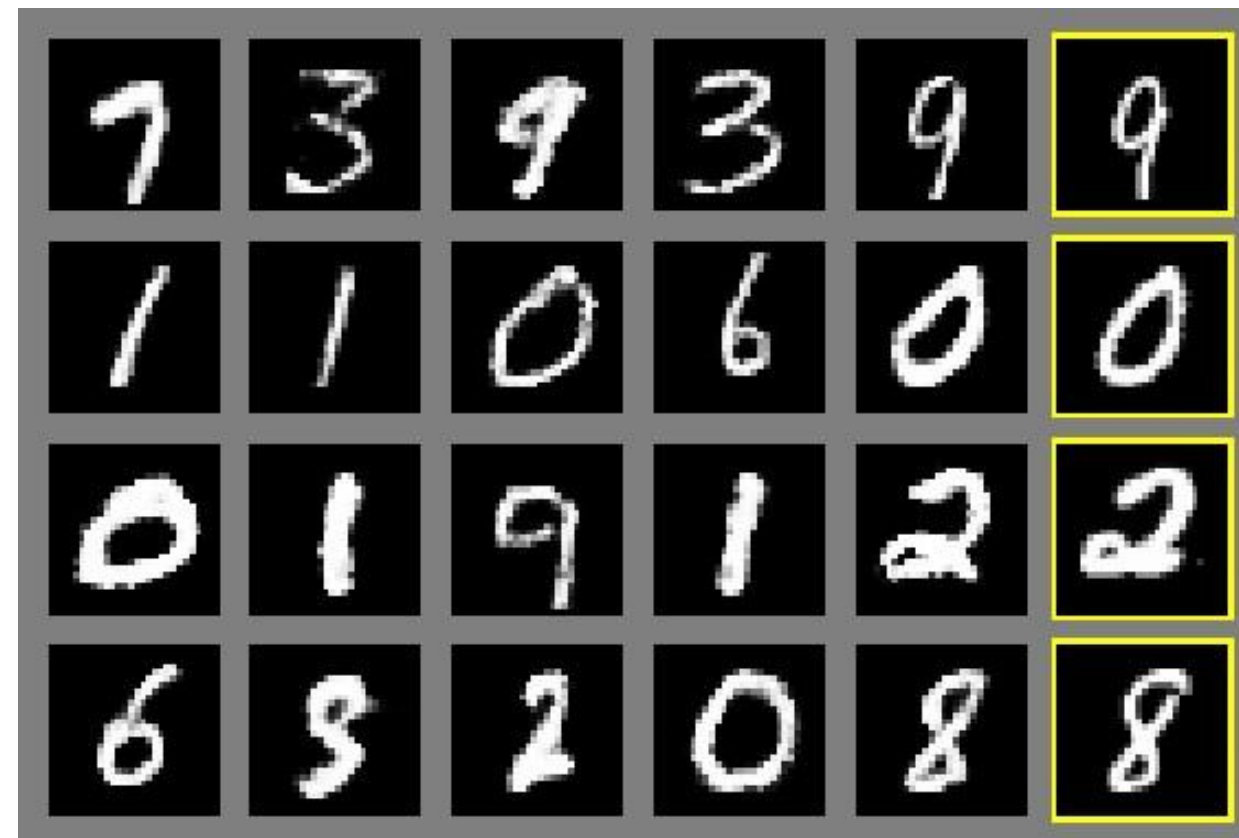


- Generator starts from Gaussian noise and generates a data point in order to fool the discriminator
- Discriminator tries to distinguish between real and fake data
- Training becomes a game between generator and discriminator
 - Solution lies in the Nash equilibrium between the two participants

GAN VISUALIZATION



RESULTS - 2016



MNIST, Toronto Face Dataset, FC CIFAR-10, Con CIFAR-10 results

RESULTS - 2021

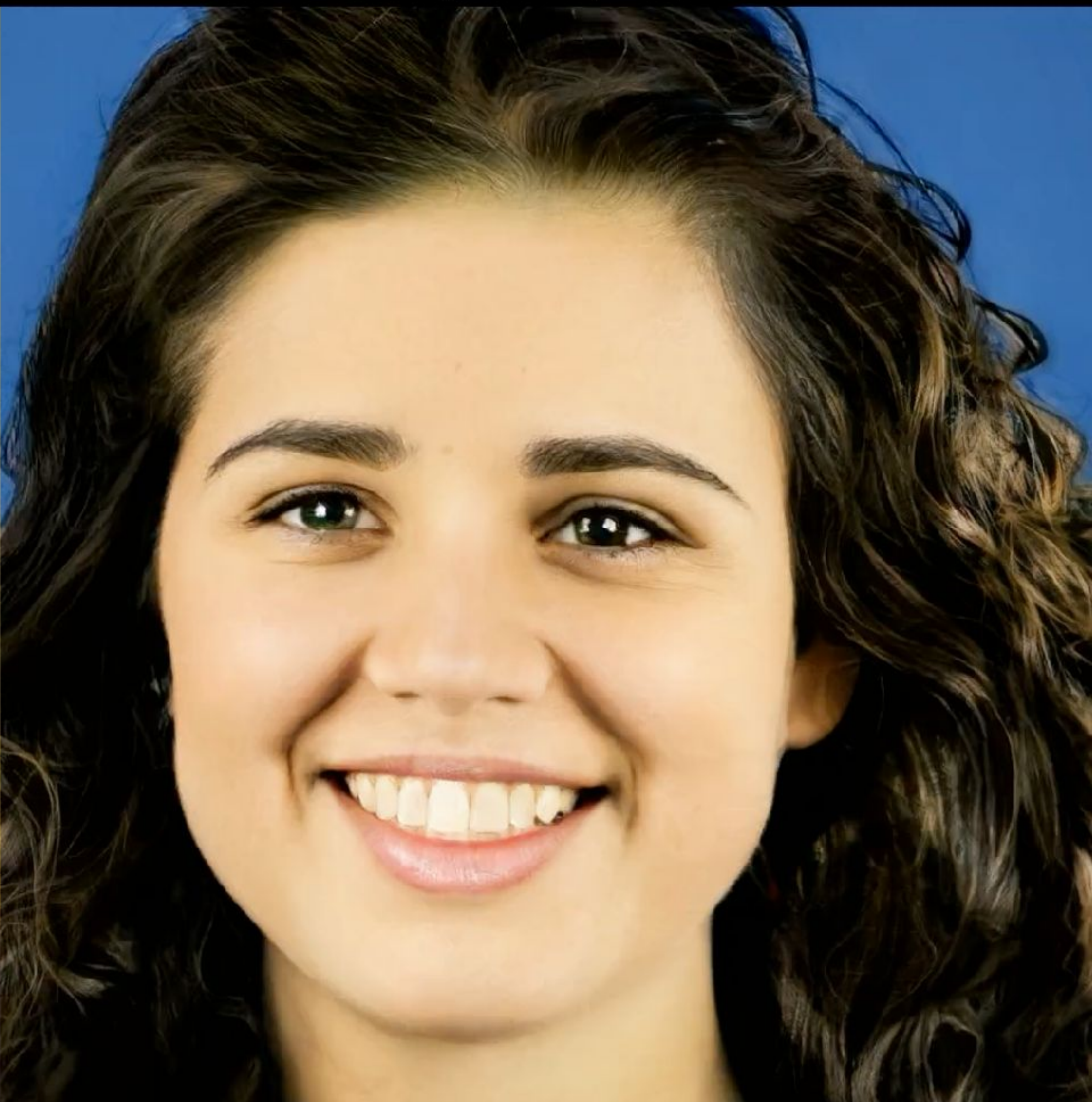
- Huge improvements thanks to several tricks, especially in the StyleGAN family
- Smooth walking in the latent space bears meaningful change in the output



StyleGAN2



StyleGAN3 (Ours)



Random latent walk using directions from StyleCLIP, GANSpace, and SeFa.



StyleGAN2



StyleGAN3 (Ours)

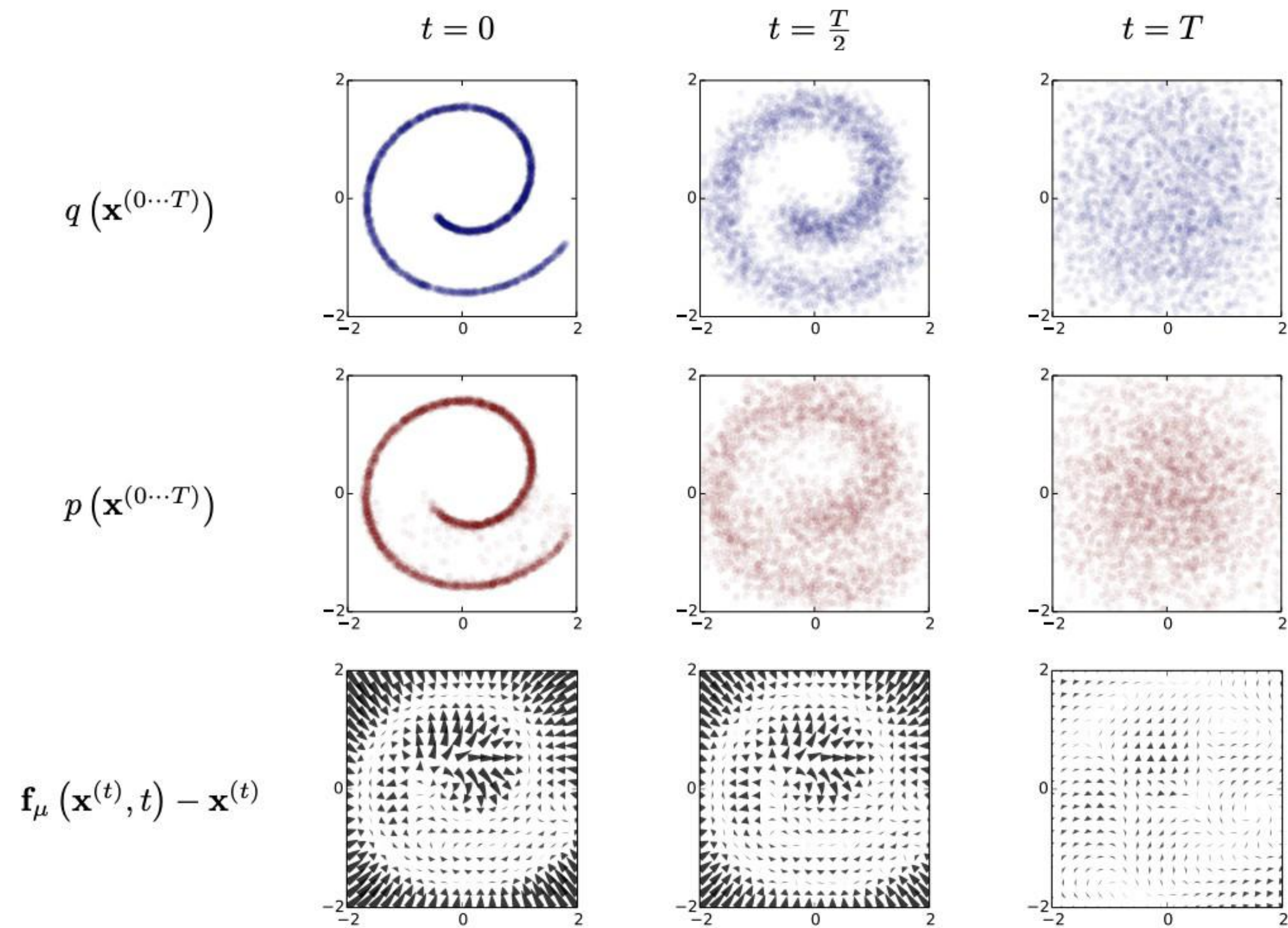


StyleGAN2

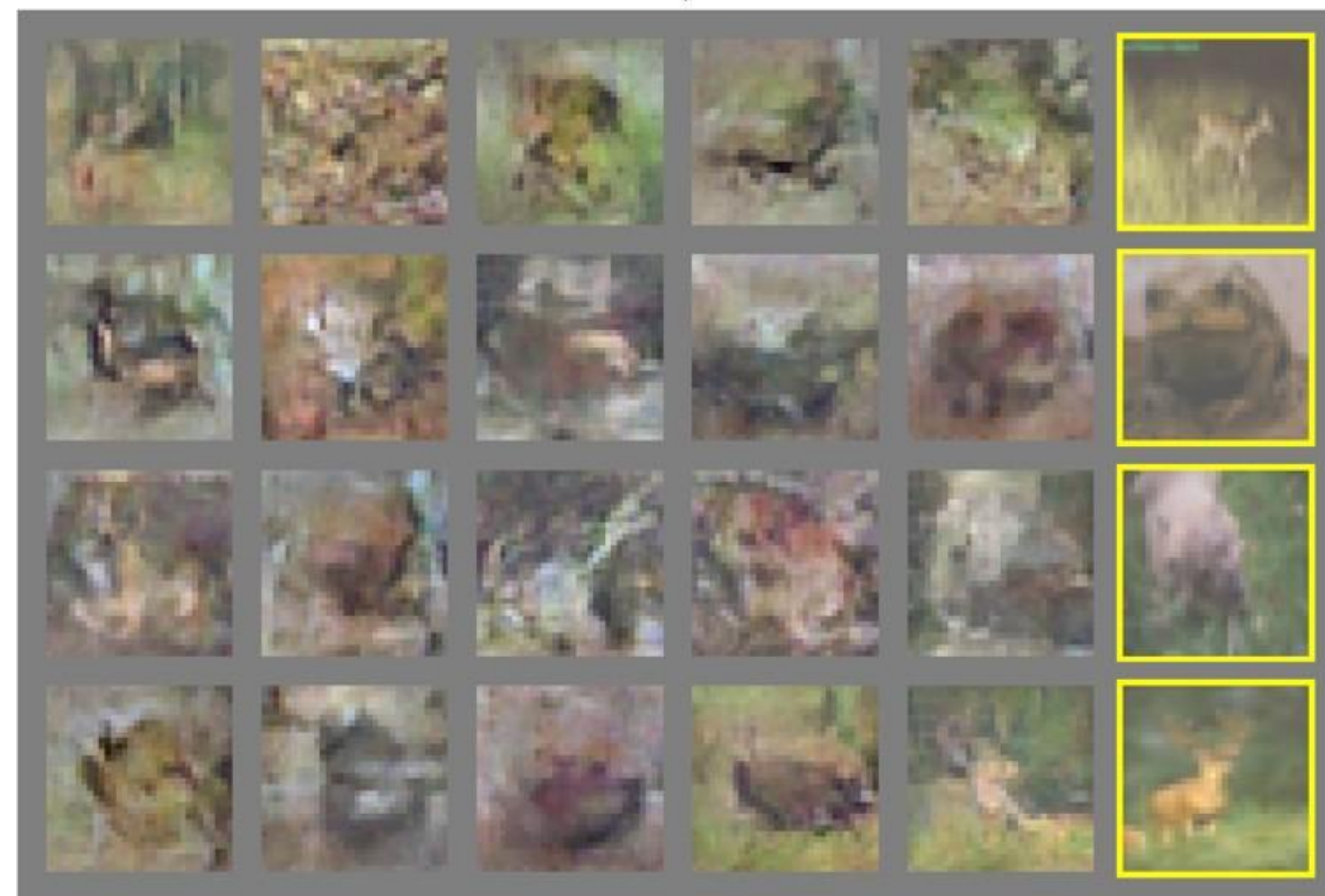
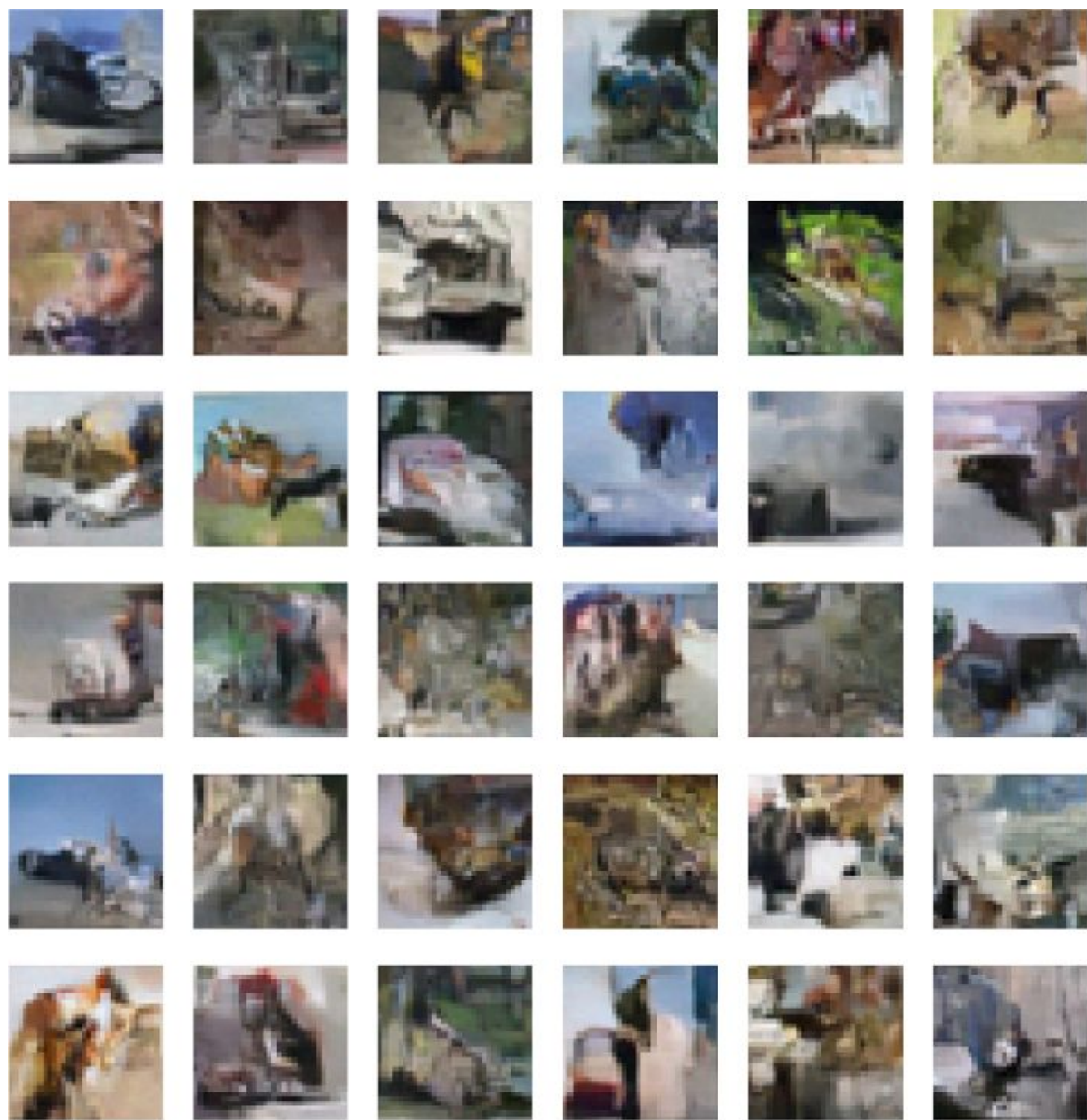


StyleGAN3 (Ours)

DEEP UNSUPERVISED LEARNING USING NONEQUILIBRIUM THERMODYNAMICS



DIFFUSION PROBABILISTIC MODELS VS GANS



Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., & Ganguli, S. (2015, June). Deep unsupervised learning using nonequilibrium thermodynamics. In International Conference on Machine Learning (pp. 2256-2265). PMLR.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2020). Generative adversarial networks. Communications of the ACM, 63(11), 139-144.

DENOISING DIFFUSION PROBABILISTIC MODEL

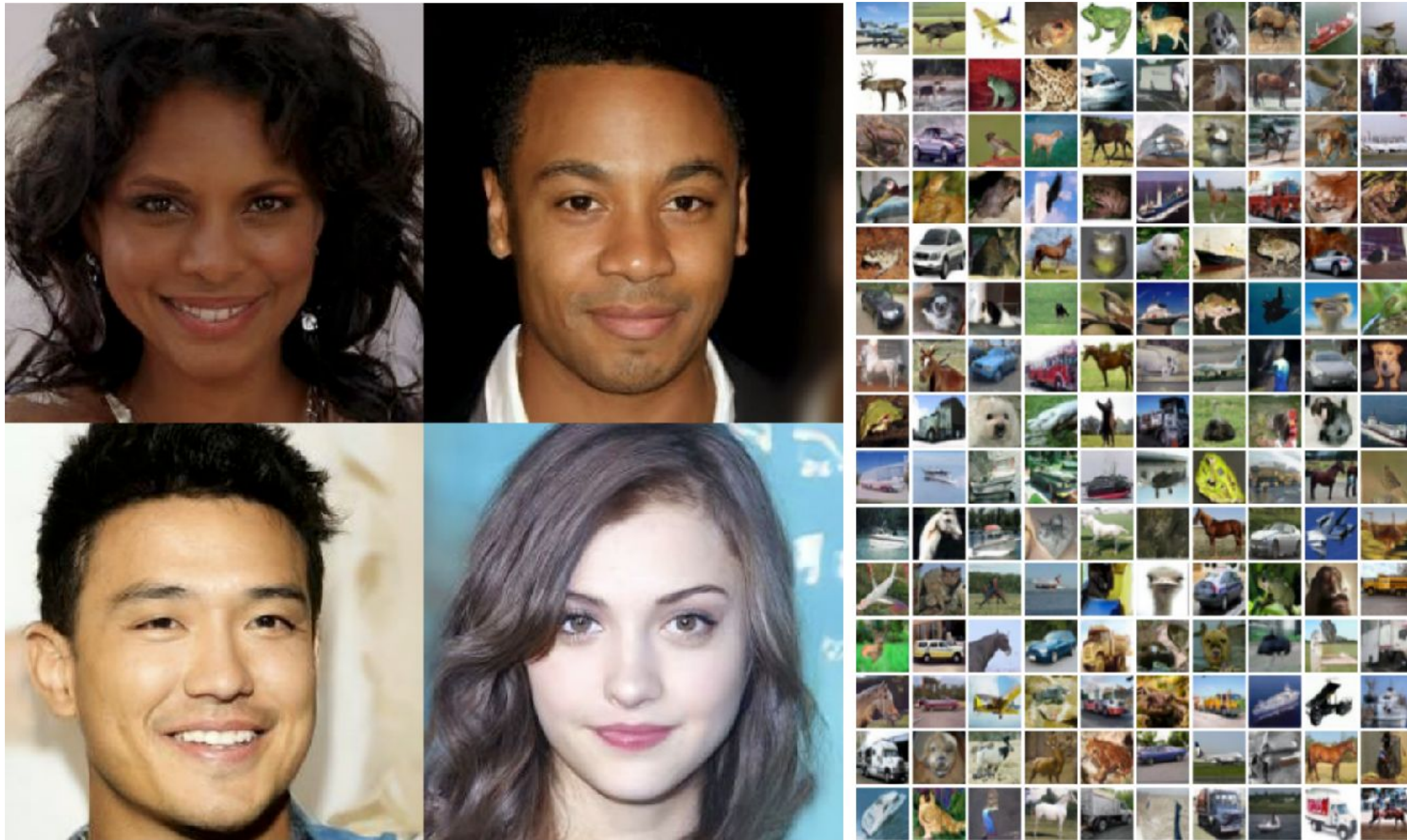
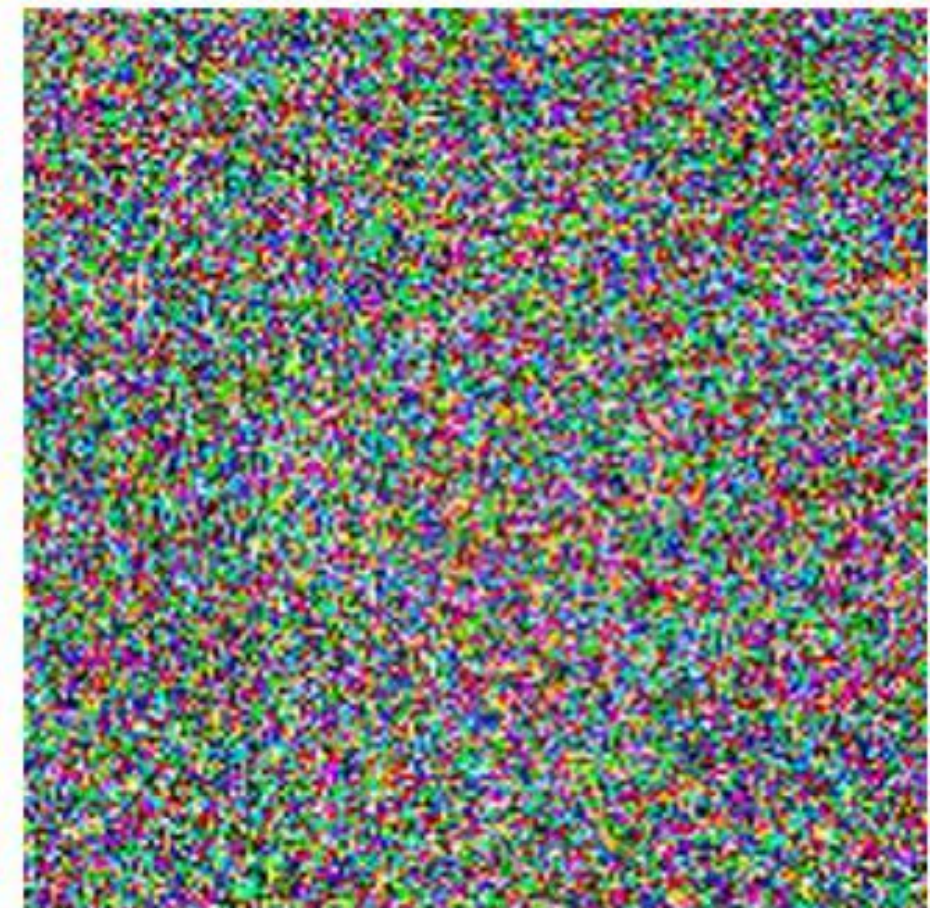
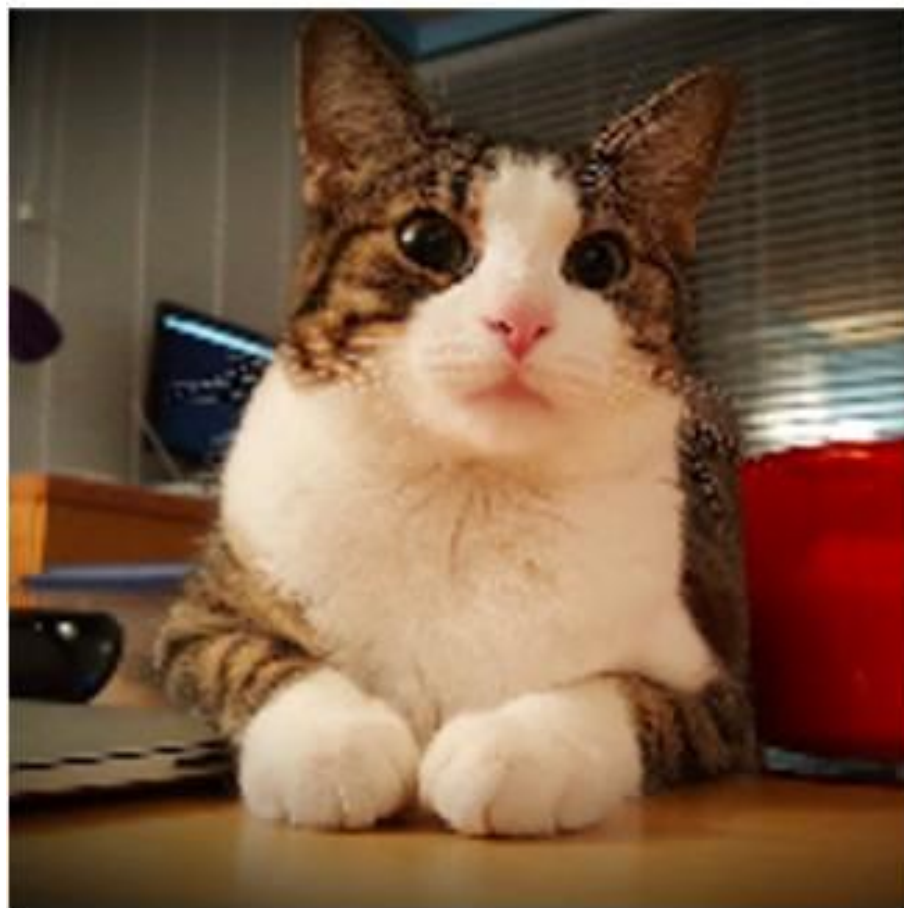


Figure 1: Generated samples on CelebA-HQ 256×256 (left) and unconditional CIFAR10 (right)

THEORY

BASIC FORMULATION

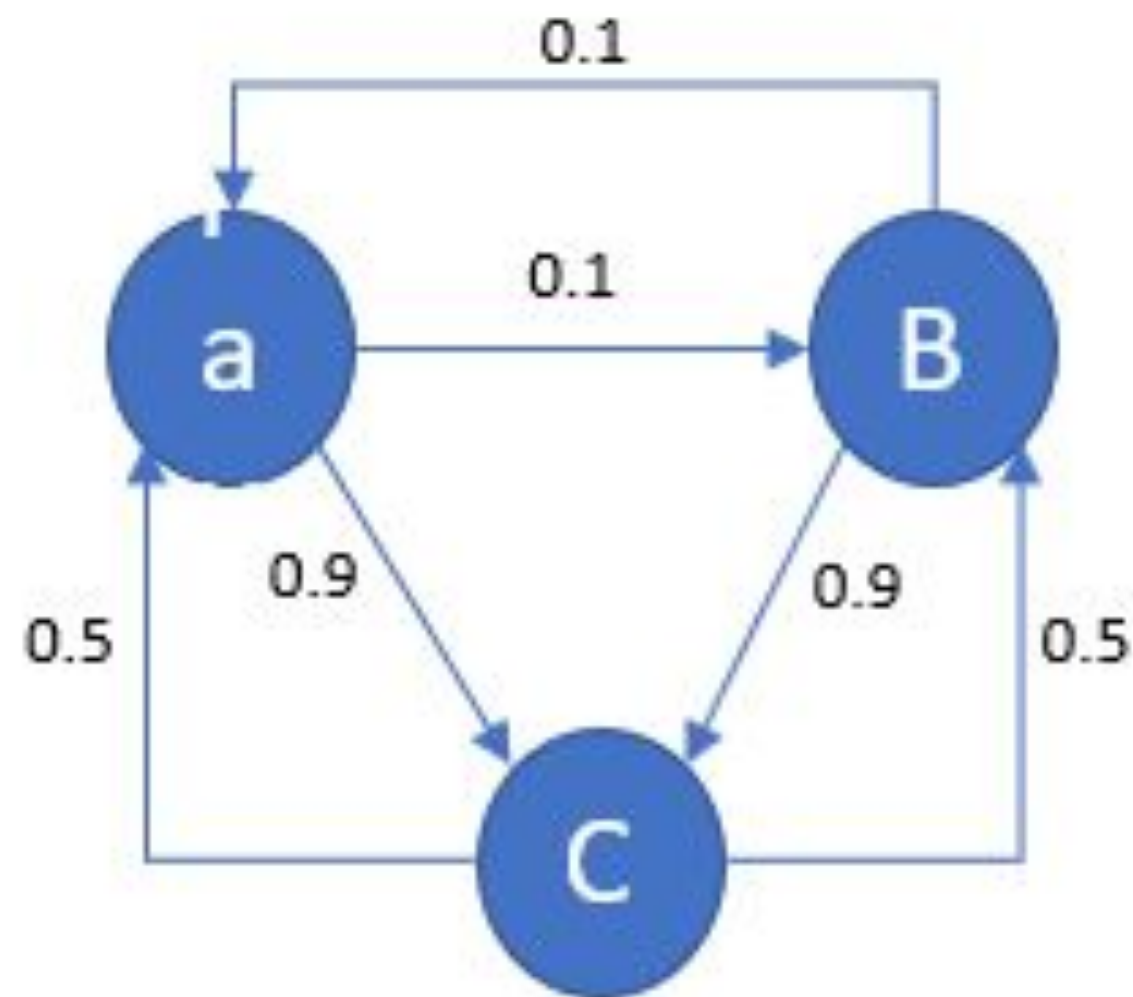


Denoising diffusion models consists of two processes:

- Forward process to add noise
- Reverse process denoises to generate data

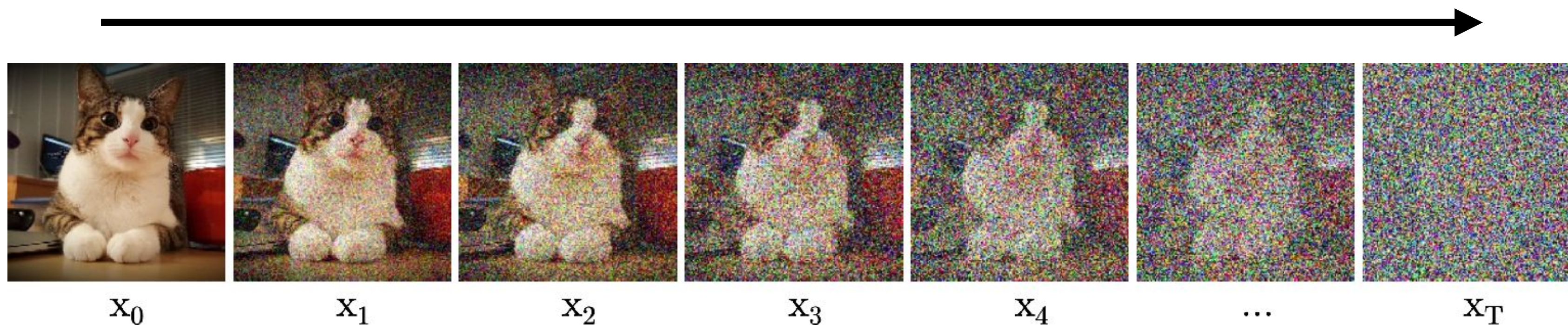
RECAP: MARKOV CHAIN

A Markov chain or Markov process is a stochastic model describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous event



$$P = \begin{bmatrix} 0 & 0.1 & 0.9 \\ 0.1 & 0.0 & 0.9 \\ 0.5 & 0.5 & 0.0 \end{bmatrix}$$

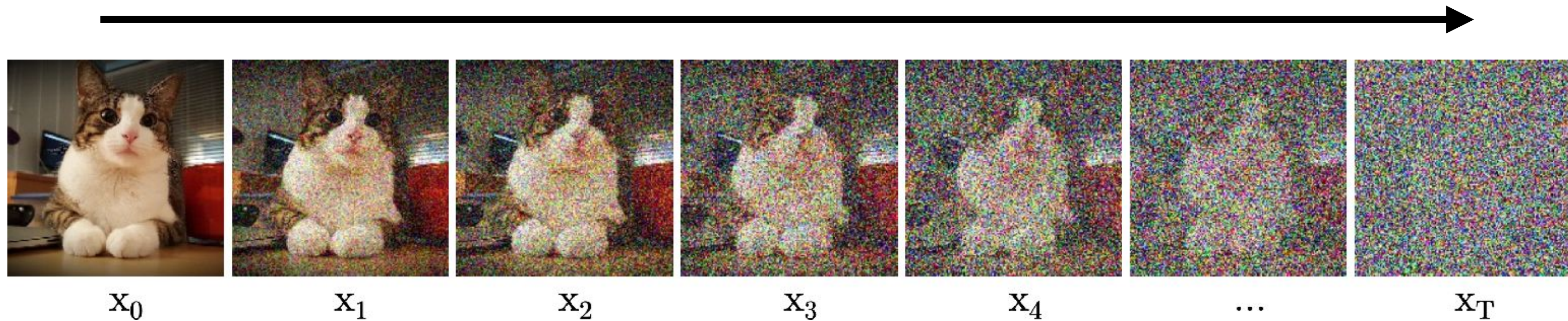
FORWARD PROCESS



Formally we have the following formulation:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}) \implies q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

FORWARD PROCESS

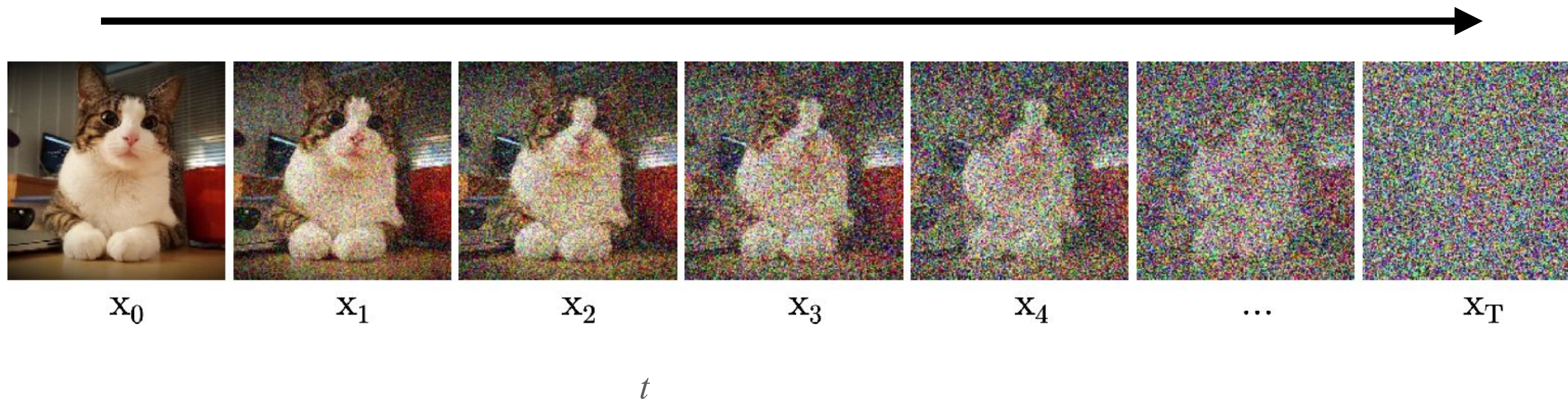


$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}) \implies q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

With a change of variables:

Define: $\alpha_t = (1 - \beta_t) \implies \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s) \implies q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I})$

FORWARD PROCESS

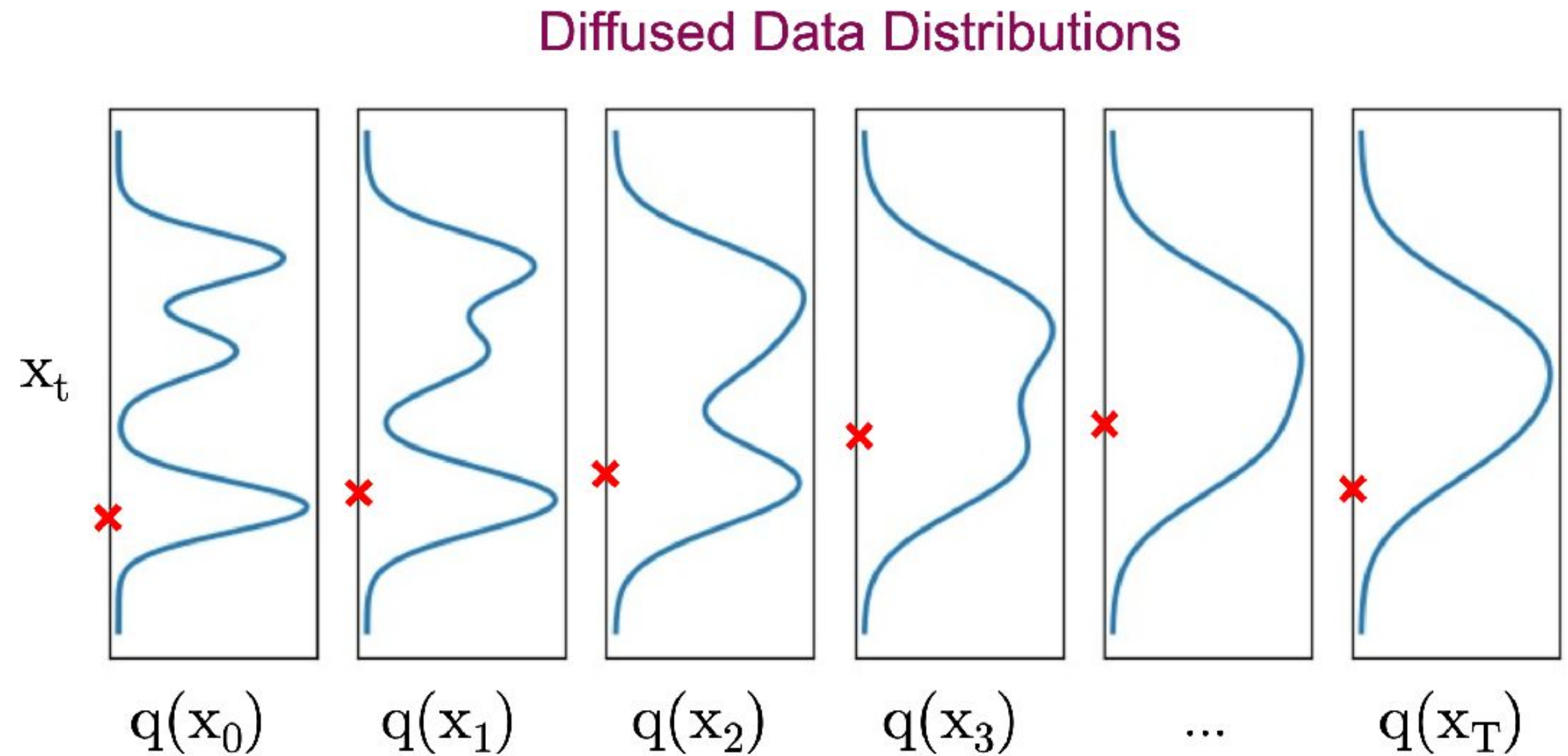


We can then sample directly at the desired timestep $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1 - \bar{\alpha}_t)}\epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I})$

β_t is the noise schedule

FORWARD PROCESS

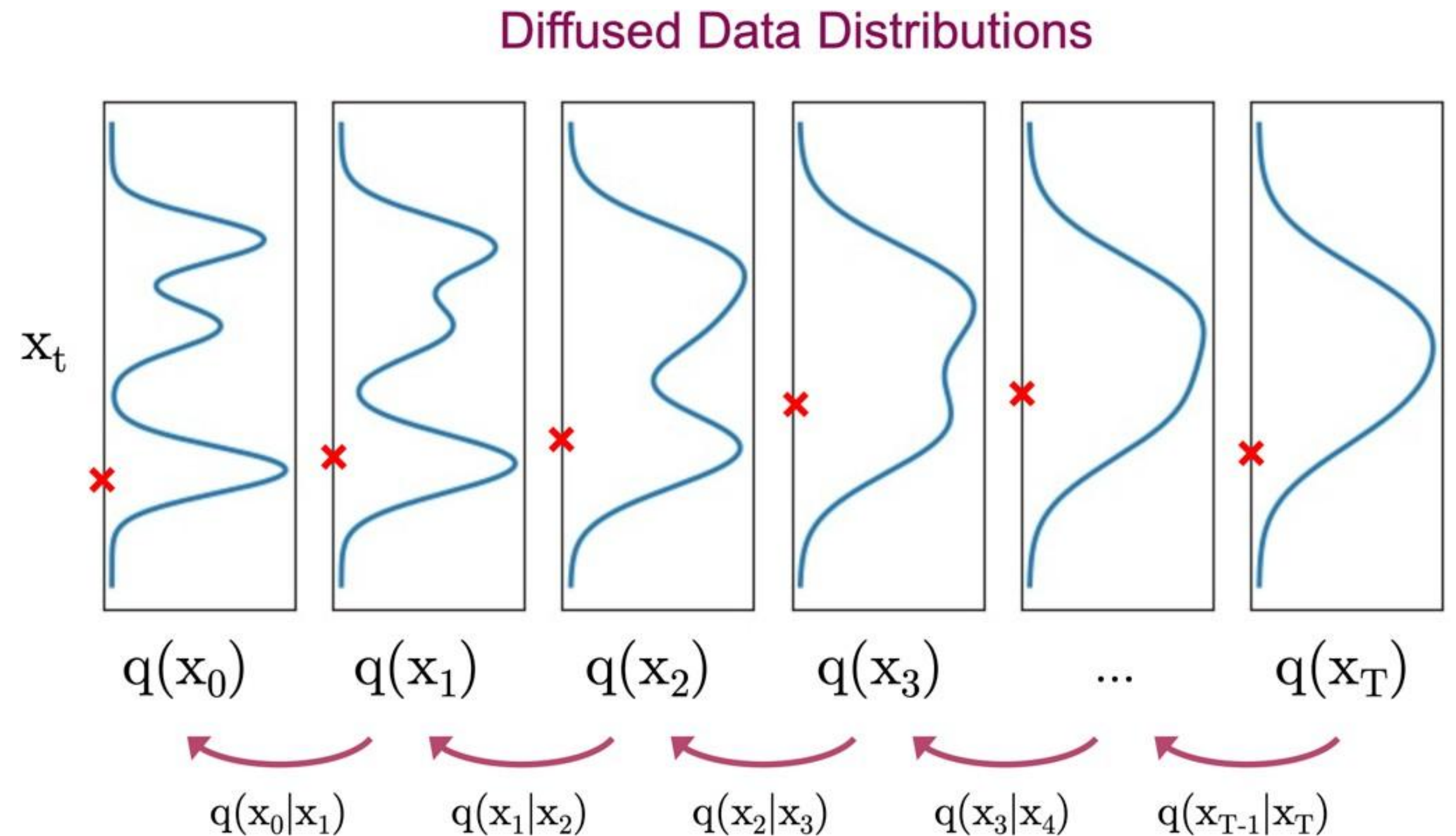
- Formally we are applying a **Gaussian convolution** to the data at each timestep
- Practically we are **smoothing** out the distribution to a Gaussian one



GENERATION PROCESS

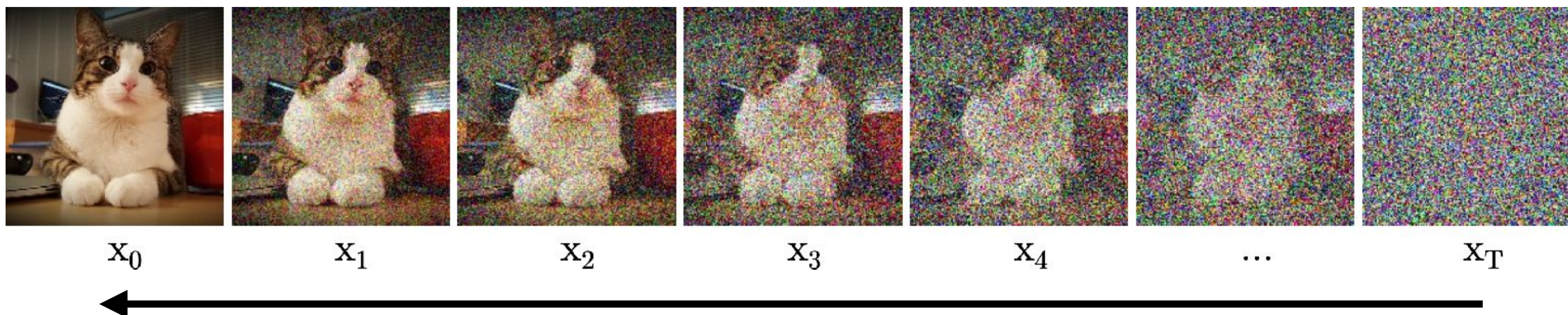
Given that $q(x_T) \approx \mathcal{N}(0, \mathbf{I})$:

- Sample $x_T \sim \mathcal{N}(0, \mathbf{I})$
- Iteratively sample $x_{t-1} = q(x_{t-1} | x_t)$



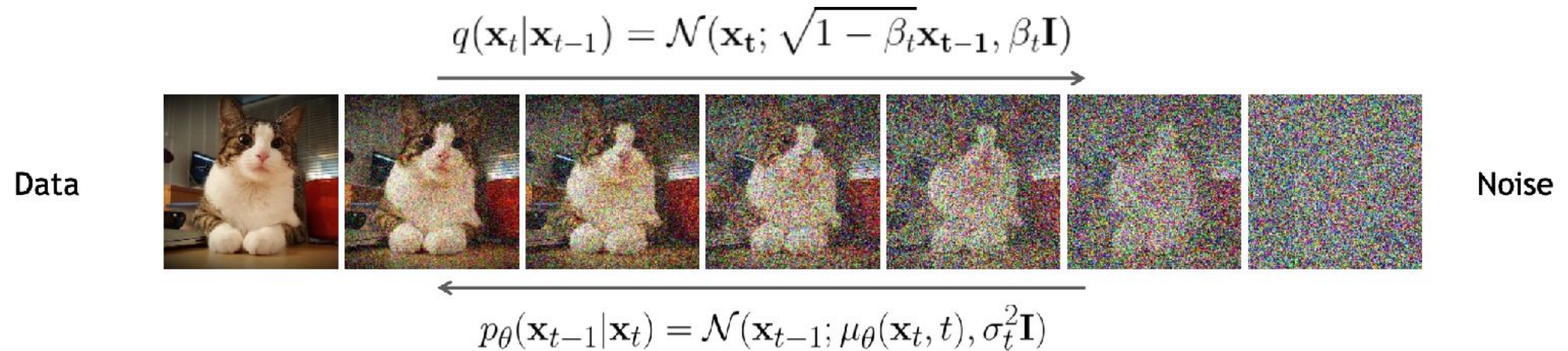
$q(x_{t-1} | x_t) \propto q(x_{t-1})q(x_t | x_{t-1})$ is generally **intractable**, but we can **approximate** with another Gaussian

REVERSE PROCESS



$$p(x_T) \sim \mathcal{N}(0, \mathbf{I})$$
$$p_{\theta}(x_{t-1} | x_t) \sim \mathcal{N}(\underbrace{\mu_{\theta}(x_t, t)}_{\text{Trainable network}}, \sigma_t^2 \mathbf{I})$$
$$p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)$$

NOISING SCHEDULE



We can control the variance of the forward diffusion and reverse denoising processes respectively. Often a linear schedule is used for β_t , and σ_t^2 is set equal to β_t .

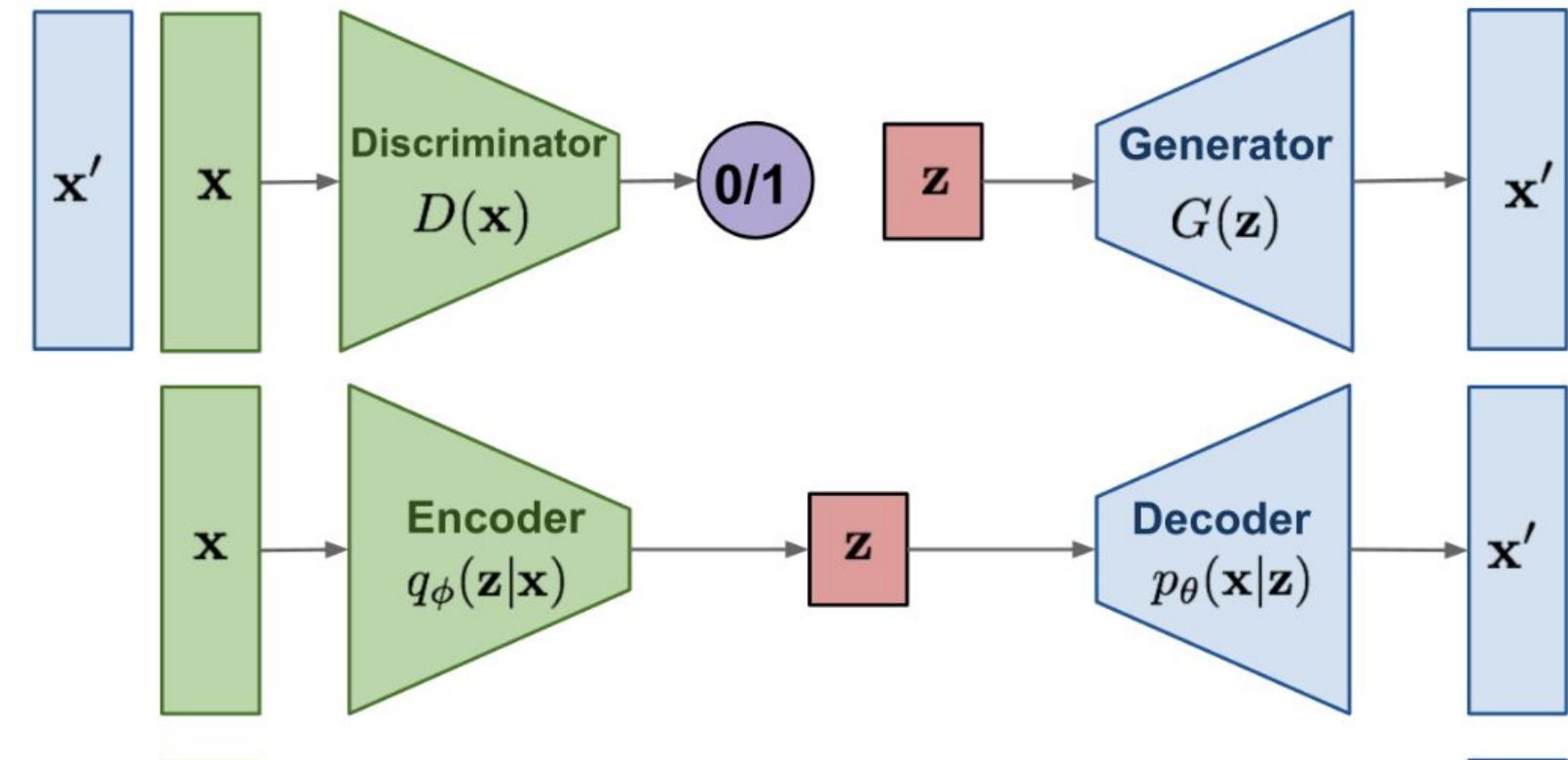
Kingma et al. [NeurIPS 2022] introduce a new parameterization of diffusion models using signal-to-noise ratio (SNR), and show how to learn the noise schedule by minimizing the variance of the training objective.

Other improvements: (Improved DPM by Nichol and Dhariwal [ICML 2021], Analytic-DPM by Bao et al. [ICLR 2022]).

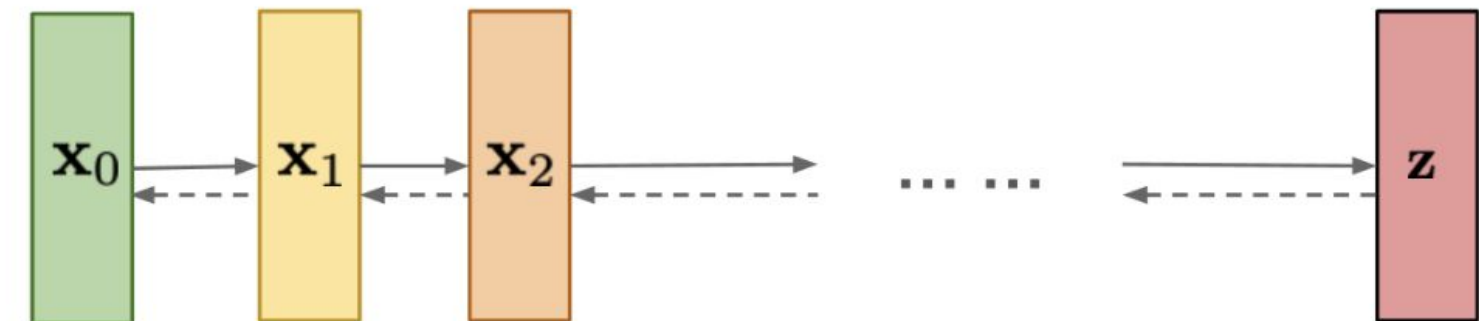
CONNECTION TO VAE, GANS

- Latent variables have the **same dimensionality** of data
- The **same model** is applied across different timesteps
- The model is trained by **reweighing** the variational bound

GAN: Adversarial training



Diffusion models:
Gradually add Gaussian noise and then reverse



TRAINING PARAMETRISATION

We can train the model in a similar fashion as VAE, with a Variational Upper Bound

$$L = \mathbb{E}_{q(x_0)} [-\log p_\theta(x_0)] \leq \mathbb{E}_{q(x_0)q(x_{1:T}|x_0)} \left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

These can be divided into three terms

$$L = \mathbb{E}_q \left[\boxed{D_{KL}(q(x_T|x_0) || p(x_T))} + \sum_{t>1} D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t)) - \log p_\theta(x_0|x_1) \right]$$

This is the original loss. Can this be simplified?

TRAINING PARAMETRISATION

$$L = \mathbb{E}_q \left[D_{KL}(q(x_T | x_0) || p(x_T)) + \sum_{t>1} D_{KL}(q(x_{t-1} | x_t, x_0) || p_{\theta}(x_{t-1} | x_t)) - \log p_{\theta}(x_0 | x_1) \right]$$

KL between Gaussians has a nice closed form, but Ho (with some math) proves the training can be simplified to a noise prediction problem

Skipping some math steps, we obtain a new loss

$$L_{simple} = \mathbb{E}_{x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0, \mathbf{I}), t \sim \mathcal{U}(1, T)} \left[|| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon, t) ||^2 \right]$$

TRAINING AND SAMPLING

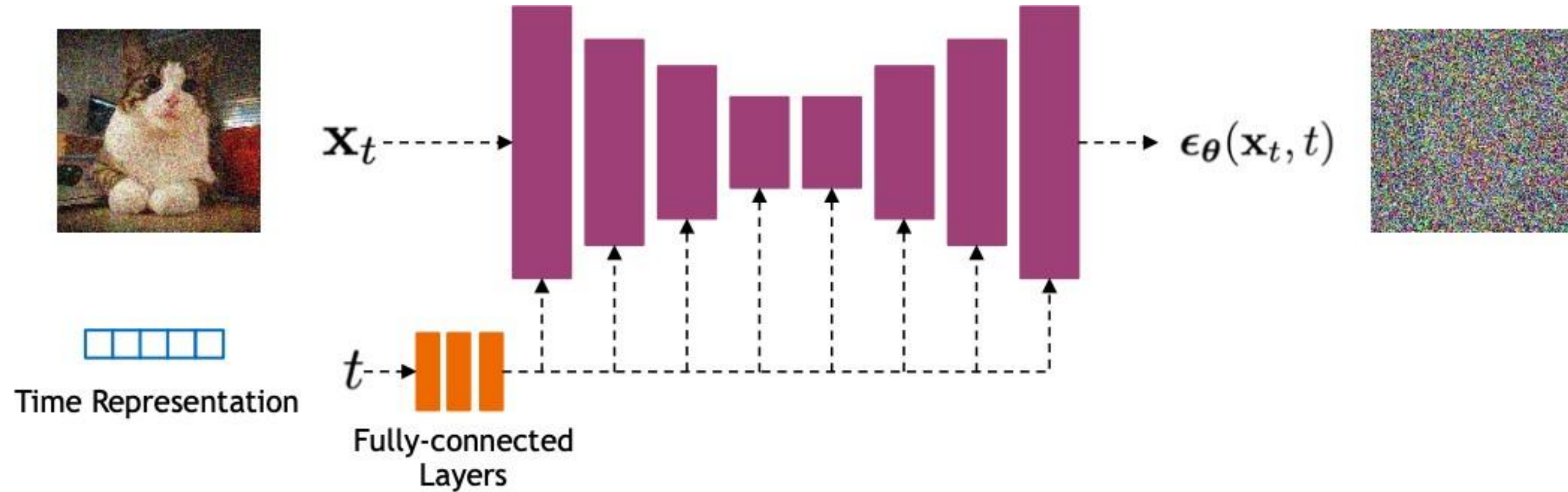
Algorithm 1 Training

```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t)\|^2$ 
6: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

NETWORK

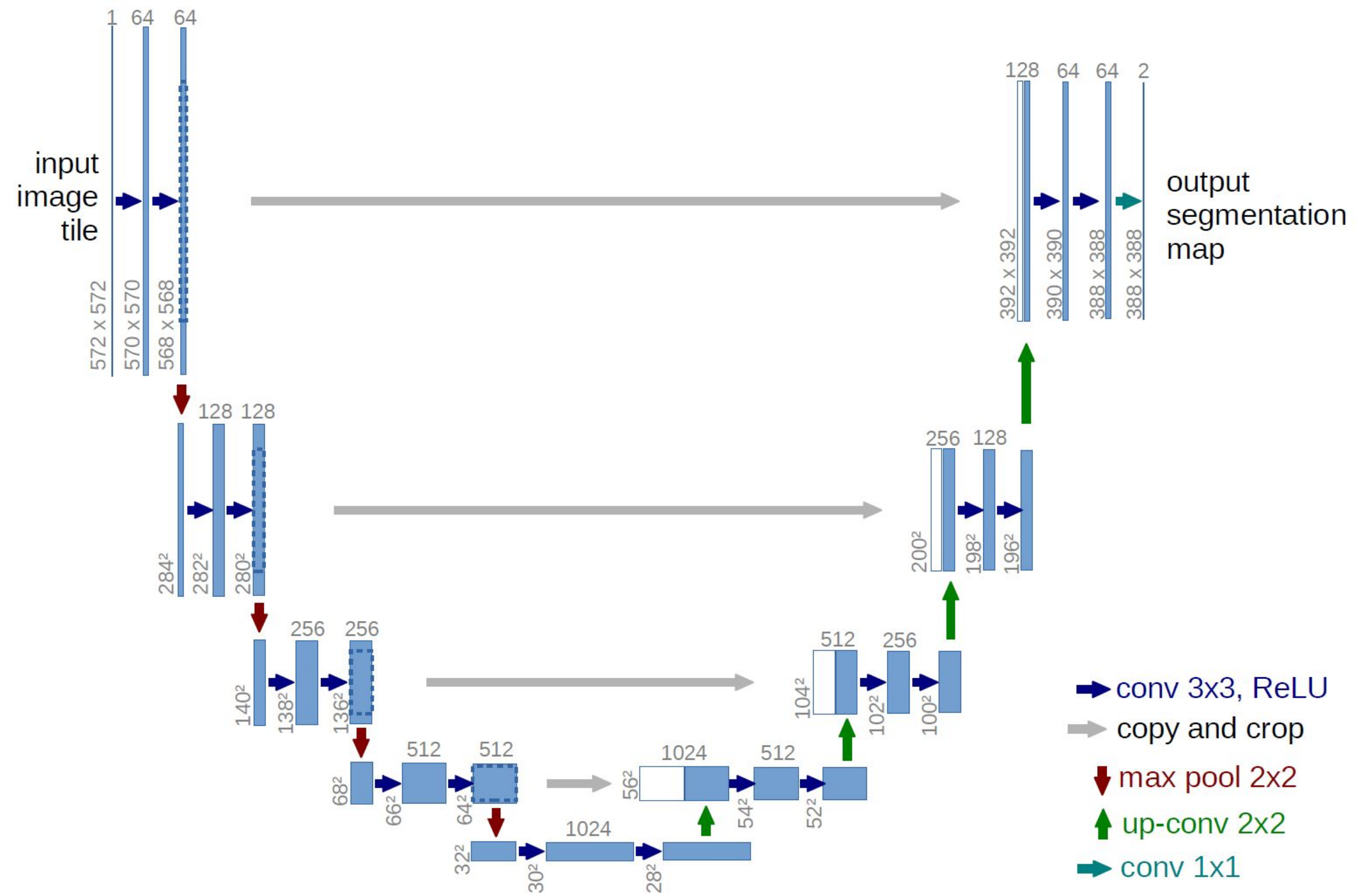


Pretty much free choice on the network architecture, no theoretical constraints on this
For images use **U-Net** (with attention)

Time features are usually **sinusoidal** of random **Fourier** features. How to embed them in the network is another free choice (e.g. spatial concatenation, AdaIN, etc)

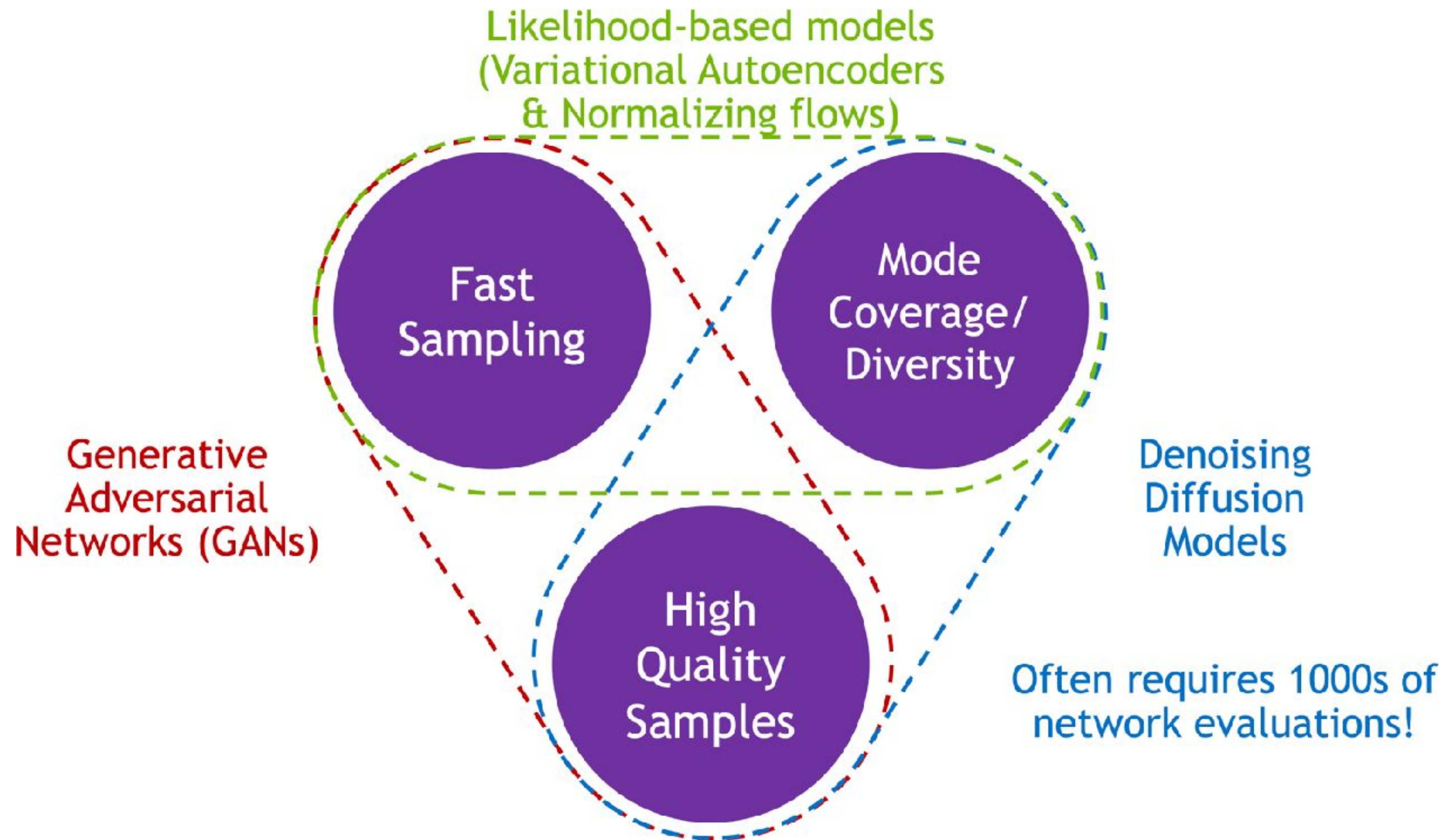
U-NET

- The U-NET architecture contains two paths.
- First path is the contraction path (also called as the encoder) which is used to capture the context in the image. The encoder is just a traditional stack of convolutional and max pooling layers.
- The second path is the symmetric expanding path (also called as the decoder) which is used to enable precise localization using transposed convolutions.
- It is an end-to-end fully convolutional network (FCN), i.e. it only contains Convolutional layers and does not contain any Dense layer because of which it can accept image of any size.



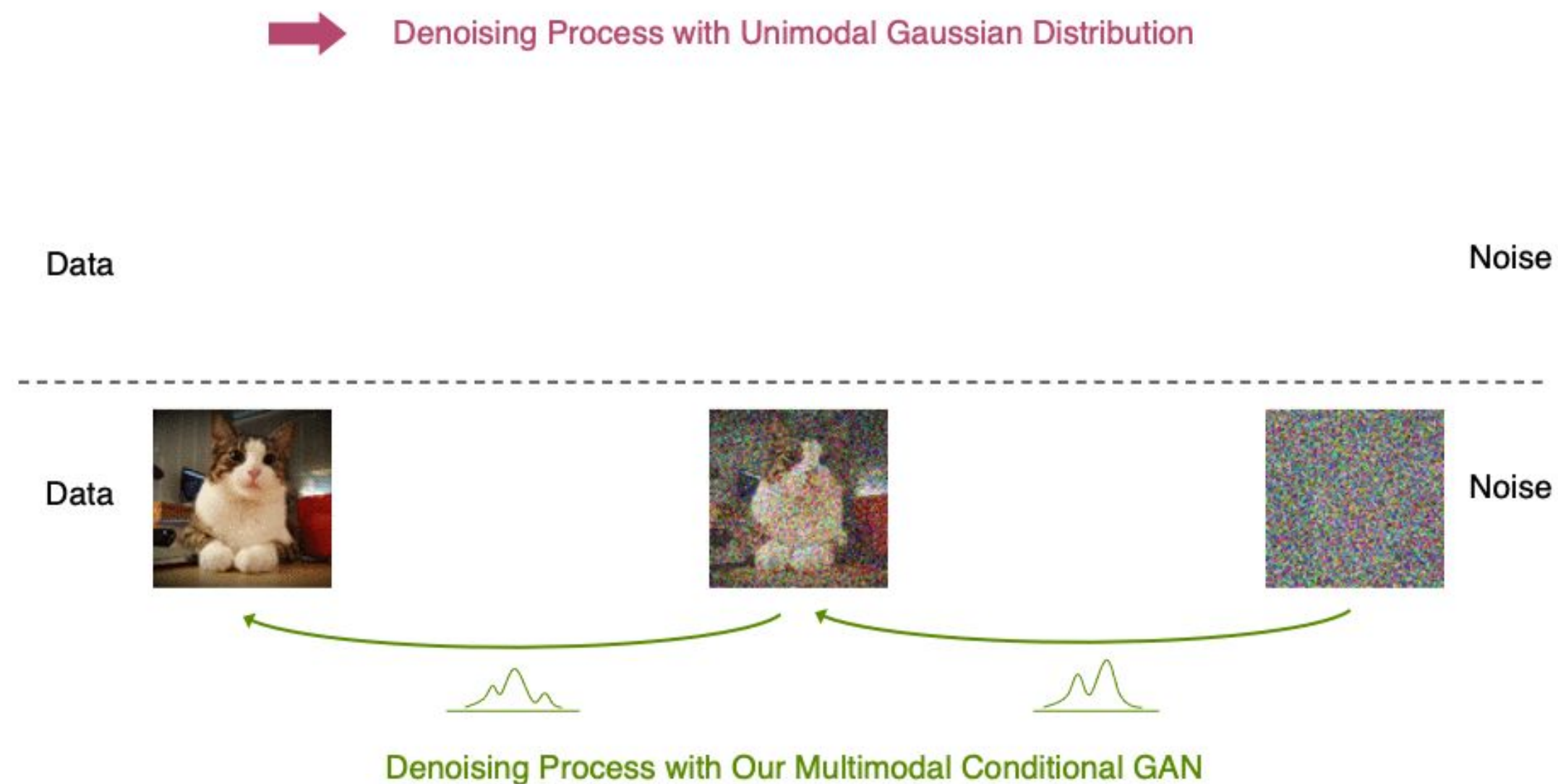
ADVANCED TRICKS

GENERATIVE TRILEMMA



DIFFUSION GANS

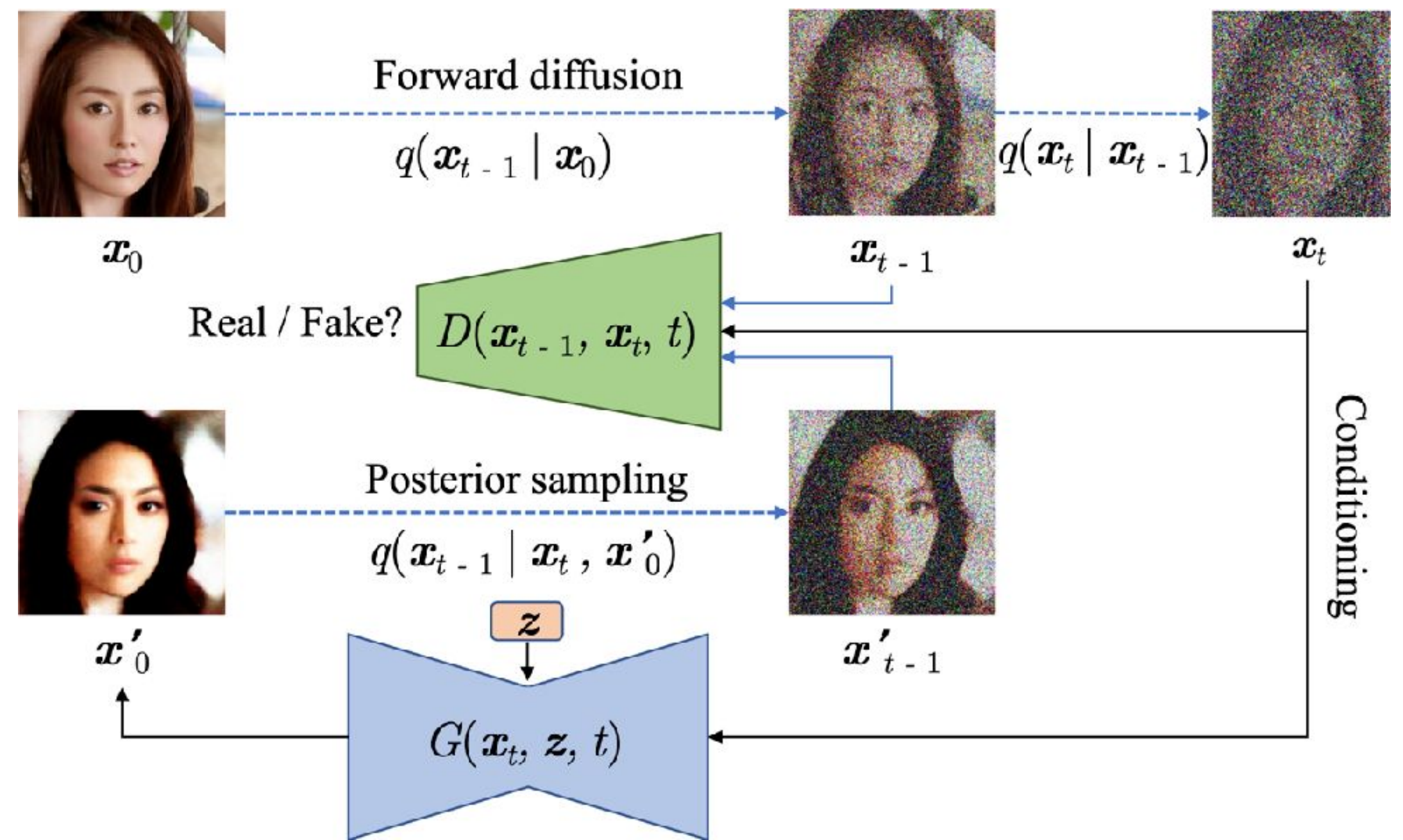
Generative denoising diffusion models typically assume that the denoising distribution can be modeled by a Gaussian distribution. This assumption holds only for small denoising steps, which in practice translates to thousands of denoising steps in the synthesis process. In diffusion GANs, the denoising model is represented using multimodal and complex conditional GANs, enabling to efficiently generate data in a few steps.



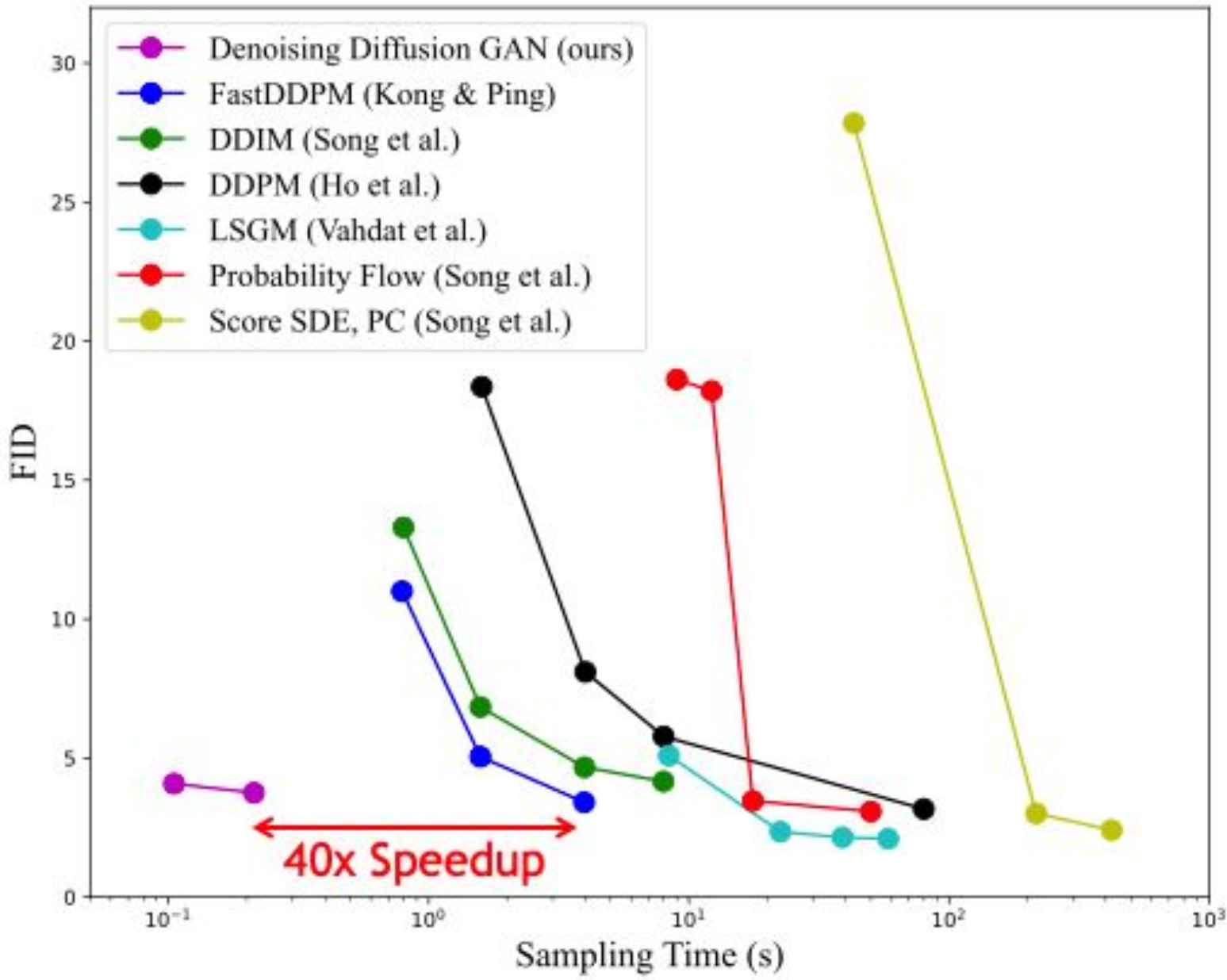
DIFFUSION GANS

Compared to a one-shot GAN generator:

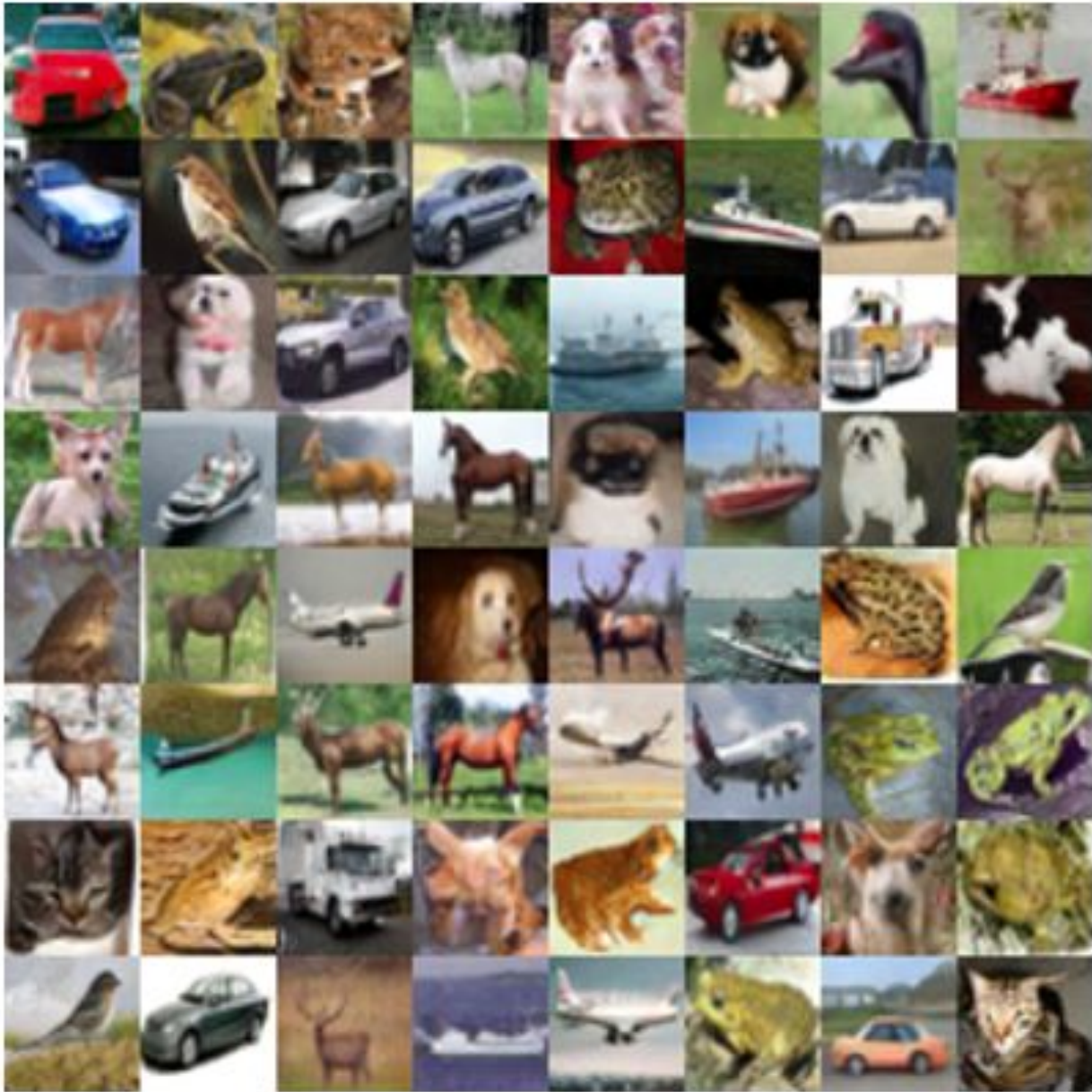
- Both generator and discriminator are solving a much **simpler** problem.
- Stronger **mode coverage**
- Better training **stability**



DIFFUSION GANS



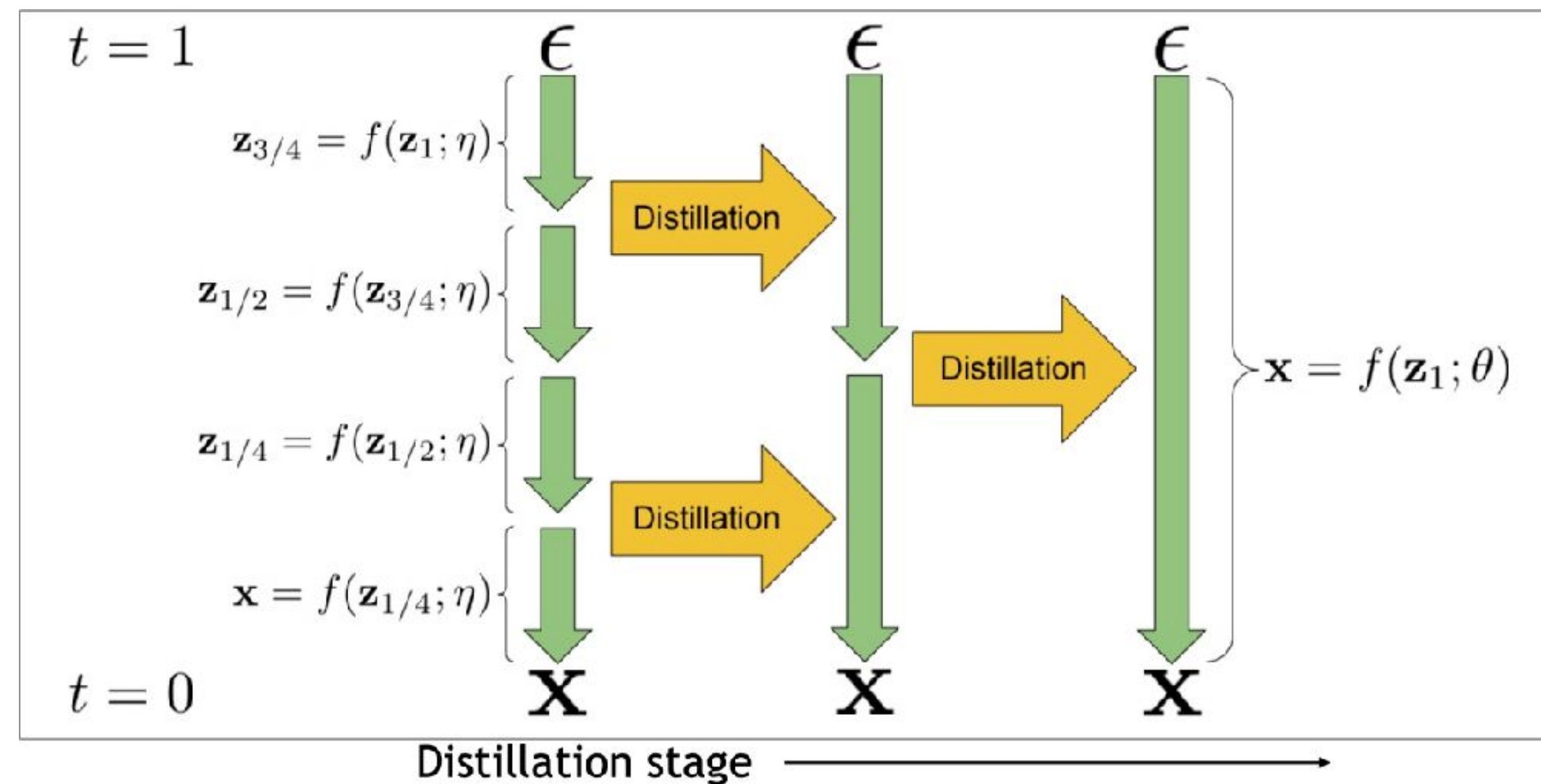
Sample Quality vs. Sampling Time



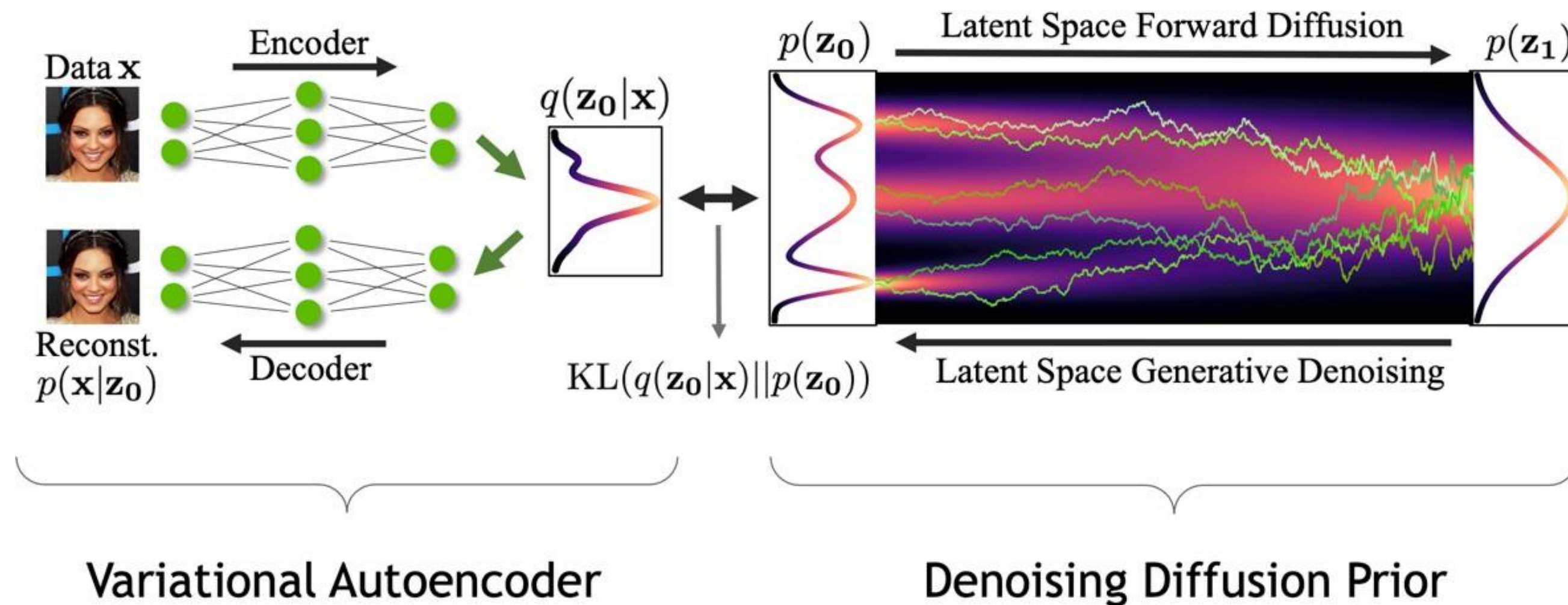
CIFAR-10 Samples

DISTILLATION

- **Distill** a deterministic DDIM sampler to the same model architecture.
- At each stage, a “student” model is learned to **distill two adjacent sampling steps** of the “teacher” model to one sampling step.
- At next stage, the “student” model from previous stage will serve as the **new “teacher”** model.



LATENT-SPACE DIFFUSION MODELS



- The distribution of **latent embeddings** is close to **Normal** distribution Simpler denoising and faster synthesis
- **Augmented** latent space
- **Tailored** autoencoders (graphs, text, 3D data, etc.)

TEXT-TO-IMAGE



“a hedgehog using a calculator”



“a corgi wearing a red bowtie and a purple party hat”



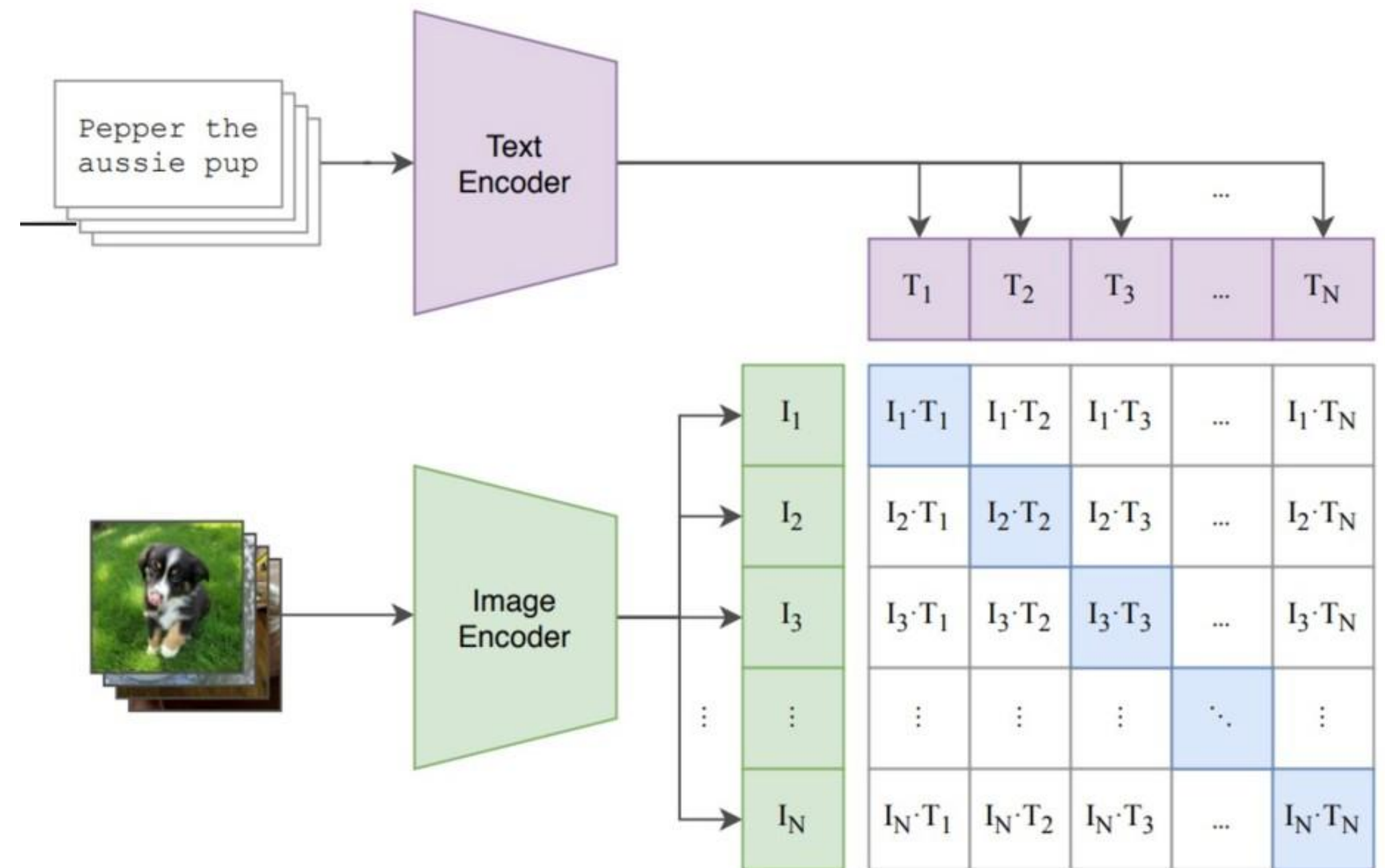
“robots meditating in a vipassana retreat”



“a fall landscape with a small cottage next to a lake”

CONTRASTIVE LANGUAGE-IMAGE PERTAINING

- Jointly train a **text** encoder and an **image** encoder
- Train by maximising the **similarity** between embeddings of (text, image) pairs
- The resulting space has **semantics** for both images and text



GLIDE: IDEA

You already understand the **Diffusion**.



$$p_{\theta}(x_{t-1} | \boxed{x_t}) = \mathcal{N}(\mu_{\theta}(x_t), \Sigma_{\theta}(x_t))$$

But you need something **More Controlled**.

Label = "Goldfinch"



Label = "robots
meditating in a
vipassana retreat"

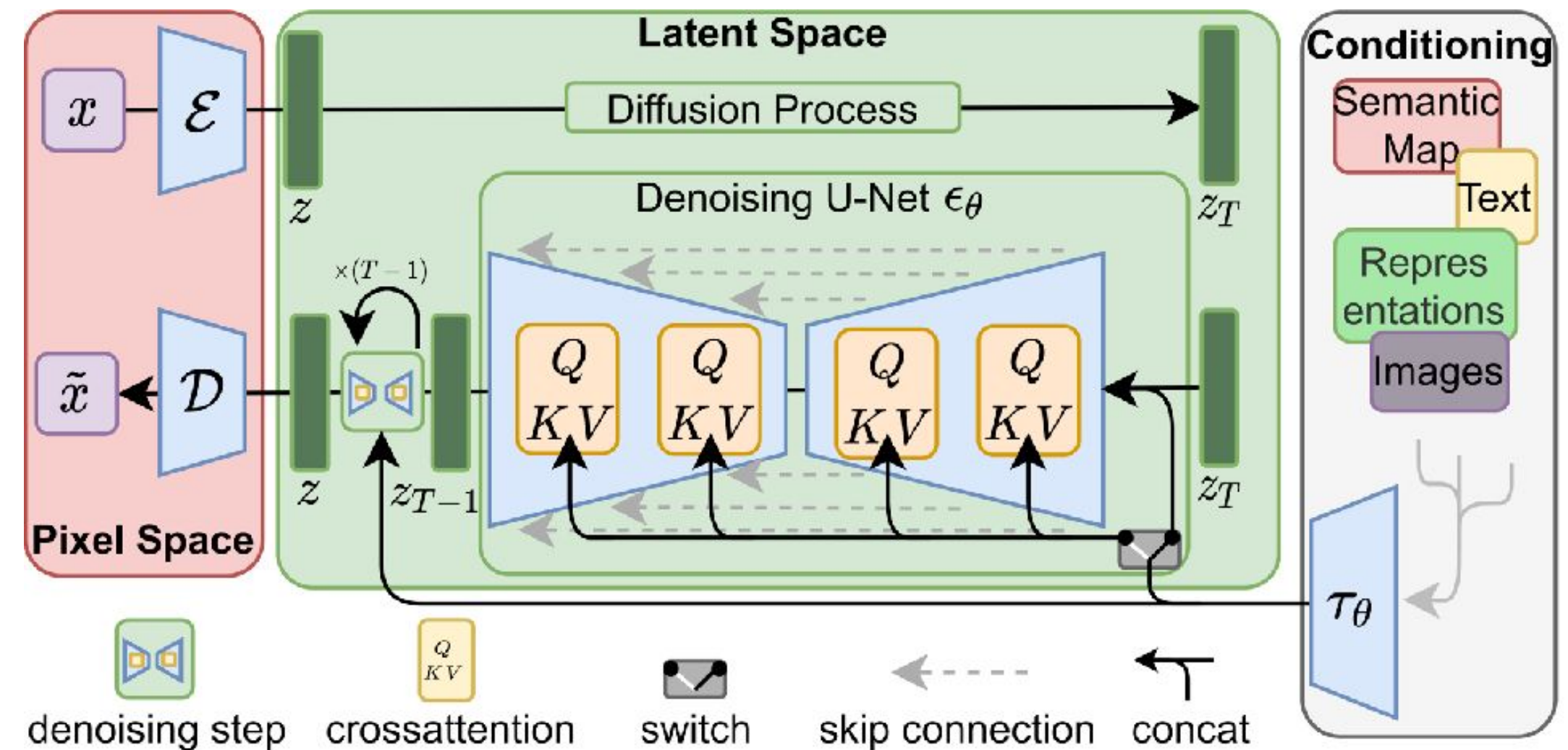


$$p_{\theta}(x_{t-1} | \boxed{x_t, y}) \rightarrow \text{Text-Conditioned Diffusion}$$

USE CLIP

TEXT-TO-IMAGE (STABLE DIFFUSION)

- Use CLIP embeddings as conditioning on **latent diffusion** via cross-attention
- Trained on a subset of LAION-5B (original dataset has **5 billion** text-image pairs)
- **Fast sampling** thanks to diffusion in latent space





DIFFUSION 100

SUPER RESOLUTION

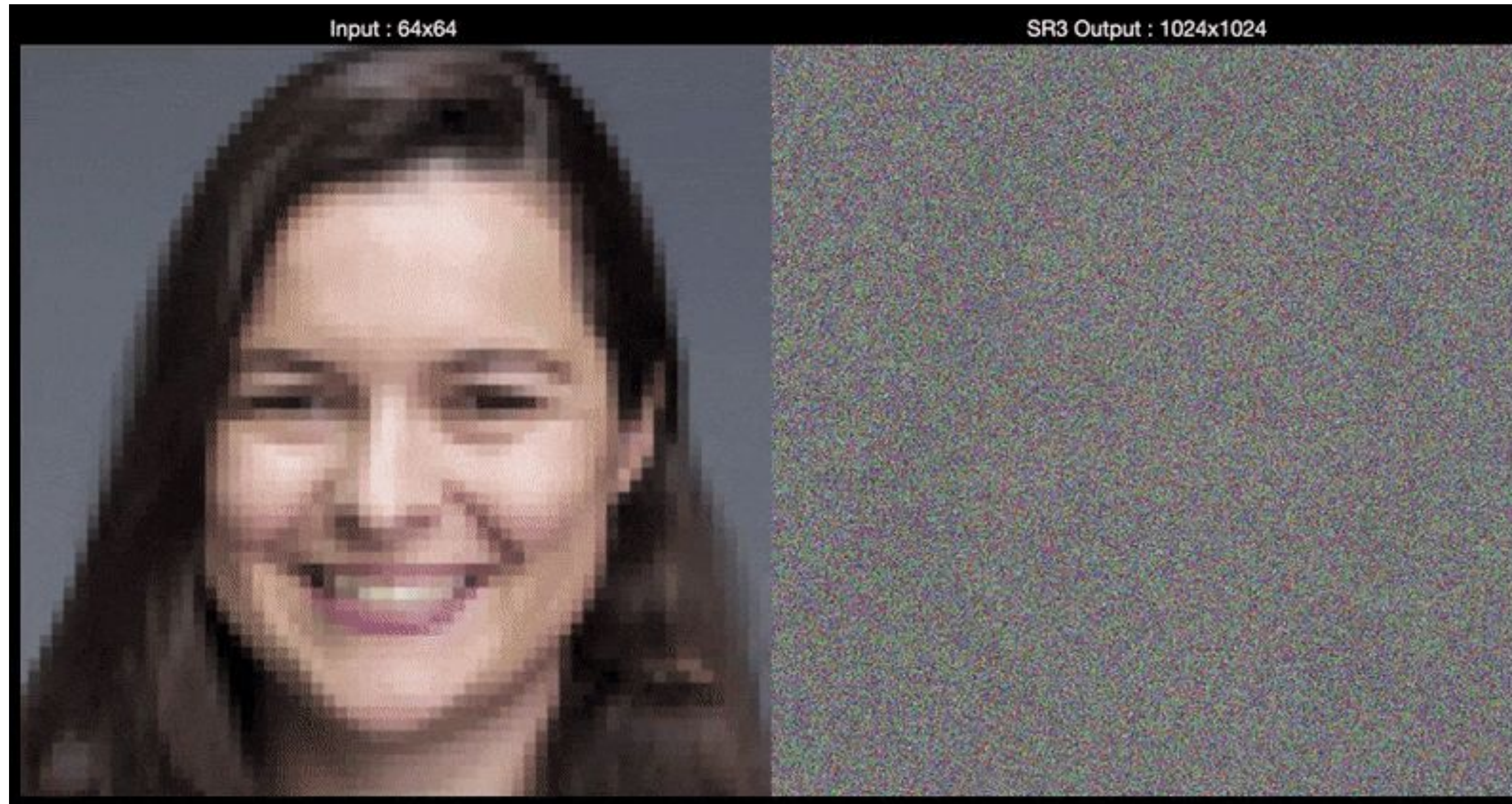
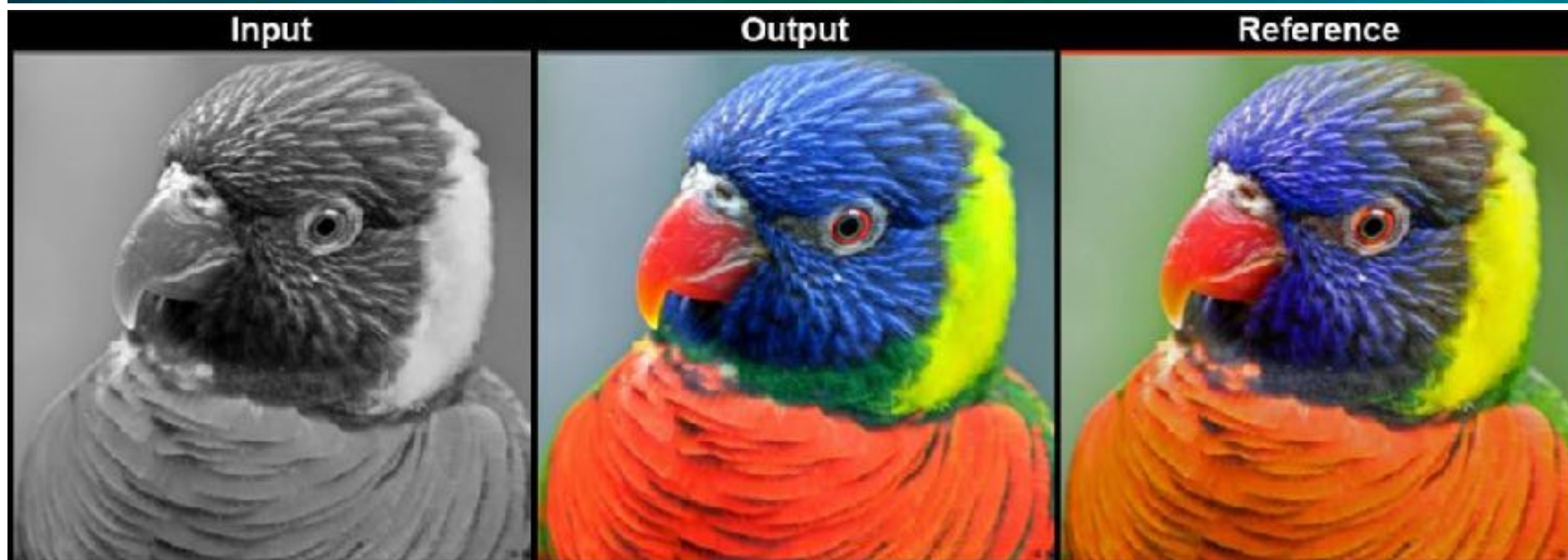
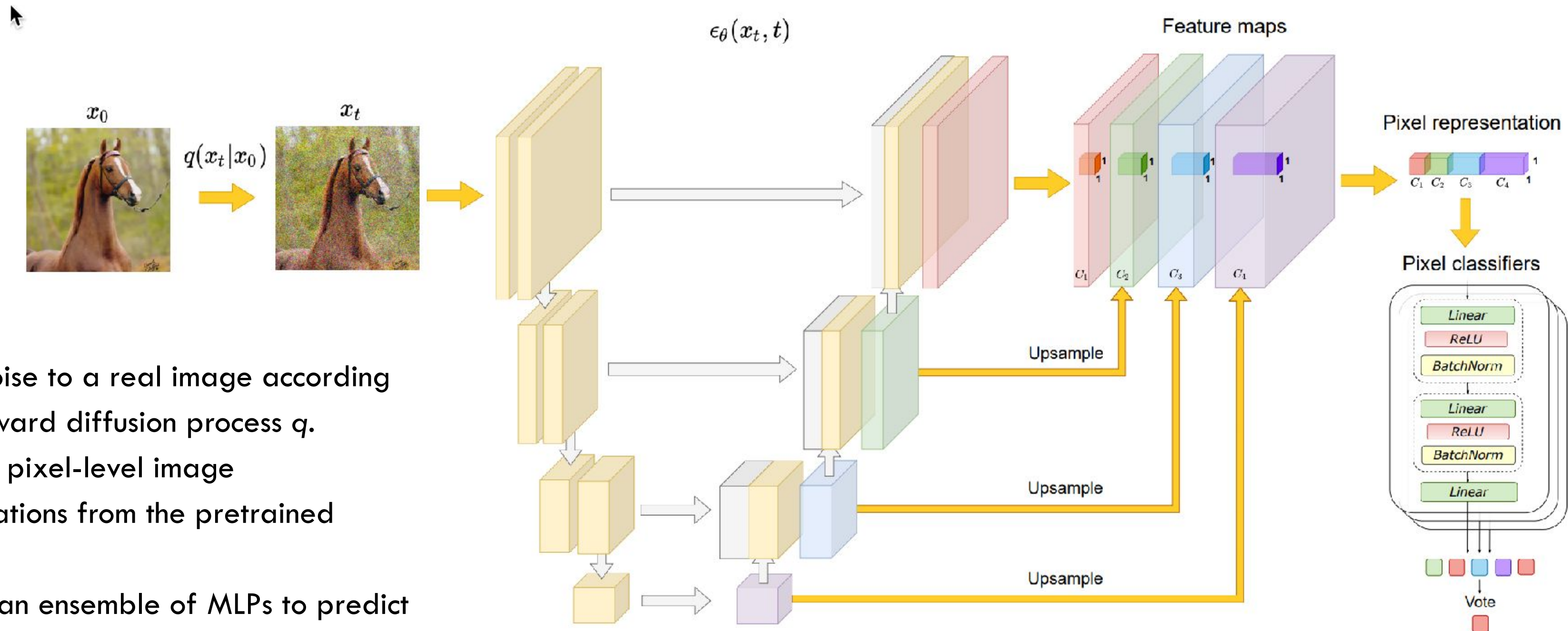


IMAGE-TO-IMAGE



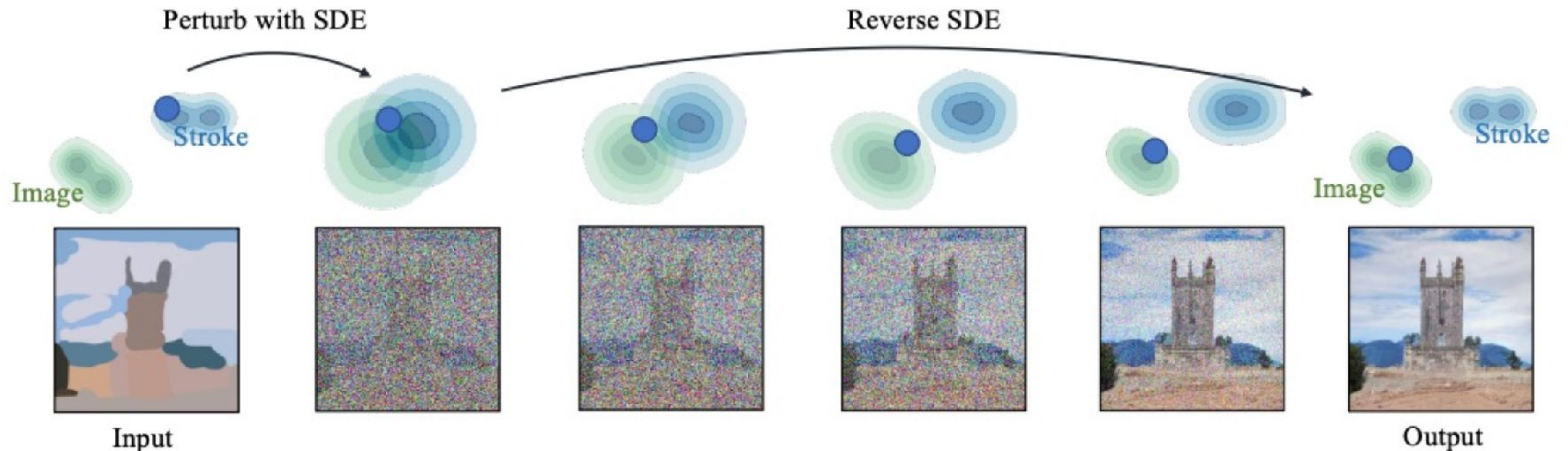
SEMANTIC SEGMENTATION



- Adding noise to a real image according to the forward diffusion process q .
- Extracting pixel-level image representations from the pretrained DDPM.
- Applying an ensemble of MLPs to predict a class label for each pixel representation.

IMAGE EDITING

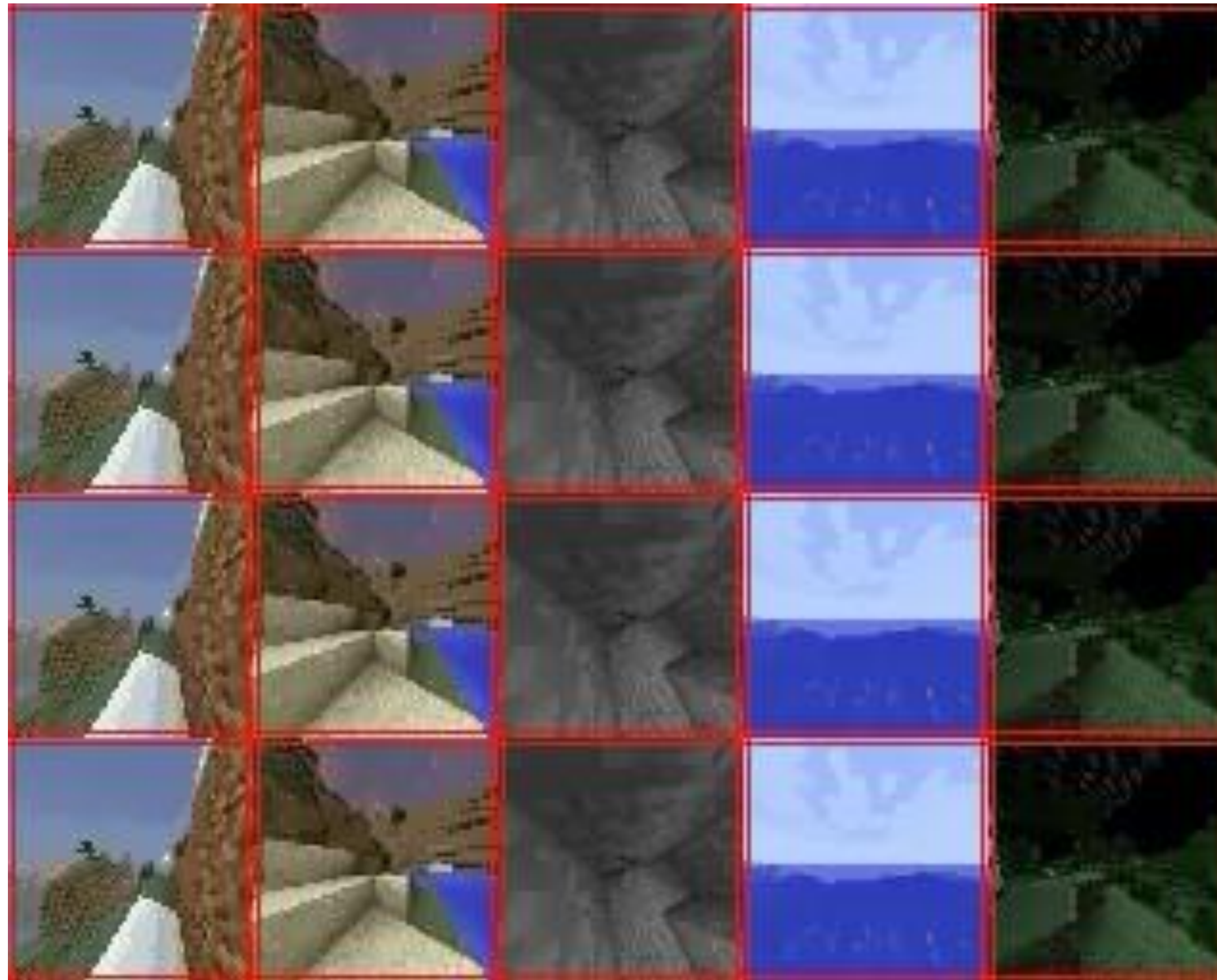
Given a stroke painting, we first perturb it with Gaussian noise and progressively remove the noise by simulating the reverse SDE. This process gradually projects an unrealistic stroke painting to the manifold of natural images.



VIDEO GENERATION



VIDEO GENERATION



3D SHAPE GENERATION



MINIMAL (SEMI) WORKING CODE

```
class Diffusion:
    def __init__(self):
        betas = linspace(sigma_min, sigma_max, num_timesteps)
        alphas = 1.0 - betas
        alphas_cumprod = cumprod(alphas)
        alphas_cumprod_prev = append(1.0, alphas_cumprod[:-1])
        posterior_variance = betas * (1.0 - alphas_cumprod_prev) / (1.0 - alphas_cumprod)

    def loss(self, x):
        e = randn_like(x)
        t = randint(0, num_timesteps, n)

        selected_alphas = alphas[t]
        x = x * selected_alphas.sqrt() + e * (1 - selected_alphas).sqrt()
        output = model(x, t)
        loss = mse_loss(output, e)

        return loss

    def sample(self, shape):
        x = randn(shape)

        for t in reverse(timesteps):
            predicted_noise = model(x, t)

            weighted_score = betas[t] / (1 - alphas_cumprod[t]).sqrt()
            mean = (1 / alphas[t].sqrt()) * (x - weighted_score * predicted_noise)
            logvar = posterior_variance[t].log() # not entirely true

            noise = randn_like(x)
            x = mean + (0.5 * logvar).exp() * noise

        return x
```

QUESTIONS?

